Deep Learning in 3D Medical Image Analysis

Jelmer Wolterink

Feb 03, 2026

CONTENTS

This Jupyter book contains notebooks for the MSc course *Deep learning in 3D medical image analysis* taught at the University of Twente.

Each cell in these notebooks is runnable, and you can load a notebook on the University of Twente Jupyter server by clicking the little rocket button on the top of the notebook. Throughout the tutorials, you will find exercises, which you can recognize as follows

---

Exercise

This is an exercise.

CHAPTER

ONE

TUTORIAL 1

1.1 February 7, 2025

Throughout the course, you will be doing a lot of Python programming and working with medical images. We have included this tutorial to get you started with some of the basics. You might find more materials in this manual for Python programming, including additional exercises.

1.1.1 Notebooks

What you are looking at right now is called a Jupyter notebook. We will use these for all tutorials in the course. A notebook is a mixture of Markdown cells (with code and figures) and code cells. There is a Python kernel running in the background that can execute your code cells, and keeps track of your variables. This is a bit like a workspace in Matlab.

There are multiple ways in which you can run a Jupyter notebook.

- You can install a Python distribution like Anaconda locally on your own computer and use JupyterLab to open this notebook in your browser.
- You can use Google Colab to run your notebook in the cloud, attached to your Google account.
- A (preferred) alternative to Google Colab is to run the notebook on the UT JupyterLab. Here, you can log in with your s-number and you get your own hosted Jupyter environment.
- You can run a notebook in an IDE like Visual Studio Code.

Start

Actually, you are now likely looking at an HTML version of the notebook. To actually be able to do something, you can either download this notebook to run it locally or somewhere else by clicking on the download button, or run it directly by clicking the little rocket button in the top right and select JupyterHub (preferred) or Colab.

To run a cell, you simply put the cursor in the cell and press Shift + Enter. For example, you can run the following cell, and (if all is well) the answer should appear below.

```
1+1
```

Throughout the tutorials, there are exercises, clearly indicated in green.

CHAPTER

TWO

PYTHON ESSENTIALS

For the tutorials, we assume that you have some familiarity with Python and its syntax. If not, there are plenty of resources available online. If you're an avid MATLAB user taking your first steps in Python, this resource might be useful.

However, to help you out a bit, we here give a brief introduction to some Python essentials.

2.1 Variables

Like in any programming language, we can store values in containers, which we call variables. For example, we can assign the value `5` to variable `a`:

```
a = 5
```

Variables can represent a number, a letter, a string, or anything else.

```
b = 2.3
course = 'DLMIA'
year = 2023
letter = 'g'
```

You can do anything you want with these variables. For example, you can add two variables to make a new variable

```
c = a + b
```

Run the cell below to check what the current value of `c` is

```
c
```

Python does not require you to explicitly declare the type of a variable. However, under the hood, variables do have a type. If you want to work with variables, be aware that they have to be of the same type.

Exercise

What happens if you try to add variables `c` and `course`. Do you understand what it says in the output?

If you are not sure which type a variable has, you can always check it using `type`, as below.

```
type(b)
```

A `float` is one way to represent numbers. Floats have decimals, and hence, Python automatically turns your variable `b` with value 2.3 into a float.

Exercise

What is the type of the variable `year`?

Exercise

types are compatible with each other. Can you figure out what type the variable has that you get when you add `year` and `b`?.

If your variable has one type but you want it to have another type, you can 'cast' it to another type as follows

```
float(year)
```

This doesn't always work, but only for types that can be cast into each other. For example, the following should give you an error

```
float(letter)
```

Variable names are case-sensitive. This means that while `b=2.3`, variable `B` is actually undefined. Try it out yourself

```
B
```

2.2 Functions

A function is a block of code that only runs when it's called. The benefit of defining a function is that - if you have to use a piece of code many times - you can easily call it. For example, we can define a function that just says 'Hello world'.

```
def greet():
    return 'Hello world'
```

If you run the cell above, you'll notice that there is no output. That is because the function is not called. To actually see some output we need to call the function like this:

```
greet()
```

Functions can also have arguments. For example, the function below takes two arguments `a` and `b` and adds these. Note that `a` and `b` are just the names of the variables within the function, i.e., within its scope. The function also has a `return` statement, to define what should be its output.

```
def add_numbers(a, b):
    return a + b
```

Exercise

Use the `add_numbers` function to add 45 and 1281.

Of course, functions can be much more complex than this. We can make new variables within a function and call one function from another function. For example

```
def multiply_numbers(a, b):
    return a * b

def my_function(a, b, c):
    d = add_numbers(a, b)
    return multiply_numbers(d, c)

my_function(2, 5, 6)
```

The sky is the limit. We can put `if`, `elif` and `else` statements in a function, make `for` loops and even call a function from within that same function (recursiveness). For example, we can easily write a function that computes the factorial of a number, i.e., $a!$.

```
def factorial(a):
    if a > 0:
        return a * factorial(a - 1)
    else:
        return 1

factorial(5)
```

One important thing to consider when working with functions (and classes) is the scope of a variable. If you define a variable in a function, it will cease to exist when the function is executed. Let's look at the simple example below. If we run this cell, we get an error which says that in the global scope, `u` is not defined.

(Note that the function below does not have a `return` statement. If we leave this out and we don't want to return anything, Python will just add `return None` for us.)

```
def manipulate():
    u = 8

u
```

Conversely, if you define a variable globally, you can read it from within a function, but you cannot change it. Let's look at a very simple example to demonstrate this. In principe, the function `change_e` should set the value of variable `e` to that of `f`. However, because `e` is a global variable, we're not allowed to change it from within this function.

```
e = 4

def change_e(f):
    e = f

change_e(6)
e
```

2.3 Objects

Another essential building block of Python are its objects. In Python, everything is an object. In practice this means that any variable you use has *attributes* and *methods*. You can easily define your own objects by defining a class. An object is nothing more than an instance of such a class. Take for example cars. There are many kinds of cars, but we can come up with a class that captures the general idea of a car with attributed and methods:

- **Attributes** Color, brand, speed
- **Methods** Accelerate, slow down

We can put this in code:

```
class Car:

    def __init__(self, color, brand, speed=0):
        self.color = color
        self.brand = brand
        self.speed = speed

    def __call__(self):
        print(f'Toot toot, I am the fastest {self.brand}')

    def accelerate(self):
        self.speed += 1

    def slow_down(self):
        self.speed = max(self.speed - 1, 0)
```

There are some important things here:

- By convention, we capitalize class names.
- Every class has an `__init__` method/function. This is a function that is called if we instantiate an object of that class. The first argument of the `__init__` function is `self`, and we can use this function to set the attribute values of the object. In this case, `color`, `brand`, and `speed`.
- The `__init__` method of this class has four arguments, but we provide a default value for `speed`, namely `0`. This means that if we call the `__init__` function (by instantiating an object), we don't necessarily have to provide a value for this argument (but we can if we want).
- This class also has a `__call__` function that allows us to call the class like a function.
- This class has two other methods: `accelerate` and `slow_down`. In these methods, we can change the attributes of the class, but only if we prepend the attribute name with `self.`.
- In the `slow_down` function, we use a so-called `built-in` function, namely `max`. This is a function that you don't have to explicitly define or import, it's just there. In this case, it makes sure that we can't have a negative speed.

Now, let's initialize Jelmer's car which is just standing still in the parking lot. We can do this by calling the class (just like we'd call a function) with arguments. This way, we indirectly call its `__init__` function. Also, we use the built-in `print` function to print its current speed.

```
jelmers_car = Car(color='gray', brand='skoda')
print(jelmers_car.speed)
```

Now, we can make the car go faster by calling its `accelerate` function, slow down using its `slow_down` function (note how the speed stays non-negative), and honk it's horn via its `__call__` function.

```
jelmers_car.accelerate()
print(jelmers_car.speed)
jelmers_car.slow_down()
jelmers_car.slow_down()
print(jelmers_car.speed)

# To call the __call__ function, we just pretend the object is a function
jelmers_car()
```

The nice thing with classes is that if we want to represent Dieuwertje's car driving on the A35, we can just make another object using the same class.

```
dieuwertjes_car = Car(color='gold', brand='hyundai', speed=100)
```

We have now created variables which are just objects of the type `Car`, similarly to variables of built-in types `float`, `int`, `str` that you have seen before. Which means that we can also write functions that take these objects as inputs. Let's define a function that determines which car would win a race:

```
def racing_time(car_a, car_b):
    if car_a.speed > car_b.speed:
        print('The first car wins!')
    elif car_b.speed > car_a.speed:
        print('The second car wins!')
    else:
        print('Everyone is a winner')

racing_time(jelmers_car, dieuwertjes_car)
```

Exercise

Use these classes and methods and change the code such that Jelmer's car wins.

There are many nice things you can do with classes, and we're not going to go into too many details here, but one thing that is useful to know is 'inheritance'. Let's say we want to make a very specific class for red Fiat cars and we're also interested in the year that they were built. We can either make a completely new class, or reuse/inherit from our existing `car` class.

```
class FiatCar(Car):

    def __init__(self, year):
        Car.__init__(self, color='red', brand='fiat')
        self.year = year
```

We can now just instantiate an object of this class and as you see below, it has all the same attributes and methods as its parent class `Car`, plus a new one, `year`.

```
my_fiat = FiatCar(1922)
print(my_fiat.color)
print(my_fiat.brand)
my_fiat.accelerate()
print(my_fiat.speed)
print(my_fiat.year)
```

Exercise

`FiatCar` is a subclass of `Car`. Write another subclass of `Car` to represent Formula 1 cars, call it `FormulaOneCar` and give it some fitting attributes and methods.

2.4 Tuples, lists and sets

Sometimes (or actually quite often), we want to store multiple values in one variable. In that case, Python has several options. We can make either a `tuple`, a `list` or a `set`. It's good to realize that these are just built-in classes, which means that they also have attributed and methods. There are some important differences between tuples, lists and sets, and depending on the situation you might want to use one or the other. Differences are nicely explained in the figure below:

- **Mutable**: Once the variable has been made, can we still change it?
- **Ordered**: Is there a particular order of the elements that is kept?
- **Indexing/slicing**: Can we ask what is in, e.g., position 3?
- **Duplicate elements**: Can the variable contain multiple elements with the same value?

A tuple is made using parentheses `a_tuple = (1, 2, 3)`, a list using square brackets `a_list = [1, 2, 3]`, and a set using curly brackets `a_set = {1, 2, 3}`.

Now let's make a tuple, list, and set and see these differences in action.

```
a_tuple = (12, 4, 8)
a_list = [12, 4, 8]
a_set = {12, 4, 8}
```

Let's first look at mutability. We can change the list and set by appending values. This works slightly differently for lists and sets, but is impossible for tuples.

```
a_list += [3, 4, 5]
print(a_list)

a_set |= {3, 4, 5}
print(a_set)
```

As you probably notice, the elements have been added to the list as well as to the set, but the order of the set has changed. A set is an unordered data structure, so it does not preserve any order we give to its elements. Instead, if we print the set, we just get its element, sorted in ascending order. For the list, the order that we give is preserved. Similarly for the tuple. This also affects indexing: because there is no order in the set, we cannot ask fo the nth item. However, we can do this for the list and tuple:

```
print(a_list[4])
print(a_tuple[1])
print(a_set[2])
```

Finally, as the table indicated, sets cannot have duplicate elements. Because there's no order, there would be no way to keep two duplicated apart. This is not a problem for the list and tuple though:

```
a_list = [1, 2, 3, 1, 2, 3]
a_tuple = (1, 2, 3, 1, 2,3)
a_set = {1, 2, 3, 1, 2, 3}

print(a_list)
print(a_tuple)
print(a_set)
```

List, tuples, and sets have all kinds of built-in functions, and can also be cast into one another. One useful built-in function is `in`. For example, if we want to check if the value `3` appears in `a_list`, we can just write:

```
print(3 in a_list)
```

2.5 Dictionaries

One last Python thing that we want to tell you (for know) is about dictionaries. Dictionaries are a very convenient way to keep tracks of *keys* and *values*. In a way, you can think of dictionaries as a set with indexing, where you can define your own kinds of indices. A typical dictionary is defined as below, and as you see, we can ask for the value of specific elements in the dictionary.

```
a_dict = {'a': 2, 'b': 4, 'c': 8}
a_dict['b']
```

For example, we can make a dictionary with the cars that we have just defined and use names as keys.

```
cars = {'Jelmer': jelmers_car, 'Dieuwertje': dieuwertjes_car}
racing_time(cars['Jelmer'], cars['Dieuwertje'])
```

There is no reason to restrict the dictionary keys or values to a specific type, we can mix and match types.

```
mixed_dict = {'Jelmer': jelmers_car, 'a': 4, 2: 'b'}
mixed_dict[2]
```

Exercise

Ask the three people closest to you right now which study programme they're in. Make a Python dictionary in which there name is the key, and their study programme the value.

CHAPTER

THREE

WORKING WITH MEDICAL IMAGES

Now let's put all this Python magic to some good use and see how we can use it to load and manipulate medical images. Medical image file formats are plenty. Most medical imaging devices store images in DICOM format, but you will also see Nifti, MetaImage (MHD), or NRRD files. What many image formats have in common is that they contain *meta information*. This is information about, e.g., the physical size of a pixel or voxel in the image, the origin of the image in world coordinates, a rotation matrix to correct for patient orientation, and even information about the CT/MRI scanner that is used and its settings. We're typically not interested in all this information or *tags*, but some of it is indispensable for image analysis.

In this tutorial, you learn to

- Load the provided images
- Extract their physical voxel size
- Convert the image to a NumPy `ndarray` object
- Visualize the image

Let's first import some of the packages used in this part of the tutorial. A package is just a collection of Python classes and functions that extend the standard built-in Python functions and classes. You will need some medical images to do this tutorial. You can download these from the SURFdrive directory. Once you have downloaded the data for Tutorial 1, add the path in the box below. Note that this is slightly different for Jupyter and Colab. In **Jupyter**, you just add the local path where you're stored the data. In **Colab**, there are several ways to use files. It might be easiest to put the data on your Google Drive.

For example, to add a path in Jupyter, do the following

```
data_path = r'/Users/jmwolterink/Downloads/Tutorial_1'
```

To add a path in Google Colab, do the following

```
from google.colab import drive

drive.mount('/content/drive')
data_path = r'/content/drive/My Drive/WHEREDIDYOUPUTTHEDATA?'
```

3.1 Python packages

Some of the packages we'll primarily use in the course tutorials are

- NumPy. This is a widely used package for linear algebra, i.e., working with vectors, matrices, and tensors in Python. These are called arrays in NumPy and you'll see them all over the place.
- SimpleITK. While there are many Python packages that will let you read a medical image or apply some functions to the image, none of them have the versatility that SimpleITK has. This is because it is based on ITK (Insight ToolKit), one of the oldest and most widely used image processing toolkits.
- Matplotlib. This package allows you to make plots and visualize images.
- PyTorch. PyTorch is one of the primary and most popular Python packages used for deep learning. It has all kinds of nice features that make it (relatively) easy to design and train a neural network.
- MONAI. MONAI is a PyTorch extension that is specifically designed for developing deep learning methods on medical images. Because it has a lot of built-in functionality, it will make your life a lot easier.

Because not all of these packages might be installed by default (depending on your Python distribution), you first need to install some packages.

```
pip install SimpleITK
pip install matplotlib
```

After that, we can import each package. This allows us to (significantly) extend the functionality of Python. Instead of just using its built-in functions, we now have access to a whole range of additional classes and functions.

```
import SimpleITK as sitk
import numpy as np
import matplotlib.pyplot as plt
import os
```

3.2 Loading an image

We first load an image that is stored in MetaImage format. In this format, an image always consists of two files: one header file (.mhd) that contains the image information, and one data file (.raw or .zraw) that contains the actual image data. The header file is human readable. We can print its contents here. Take a look at the attributes that are included in the meta information for a typical image.

```
image_file = os.path.join(data_path, r'TEV1P1CTI.mhd')

with open(image_file) as f:
    for line in f:
        print(line.strip())
```

Exercise

Infer from the header information if this is a 2D or a 3D image.

There are all kinds of tags in this header file that are relevant for the way in which the image is read. However, the nice thing is that we can also directly read this header file into a SimpleITK Image object. The image reader will read all meta information into the Image object and also load the pixel values from the data file.

```
image = sitk.ReadImage(image_file)
```

You can then use methods in this object to retrieve the properties that are in the header file. For example using the following queries.

```
print(image.GetSize())
print(image.GetOrigin())
print(image.GetSpacing())
print(image.GetDirection())
```

Now we know that the image has a particular size, we can also find the value of a pixel in that image. For this, we can use the GetPixel method, and we can use SetPixel to change values.

```
print(image.GetPixel(100, 100, 10))
```

```
image.SetPixel(100, 100, 10, 0)
```

Of course, setting and getting individual pixels is all quite cumbersome and this is not the optimal way to work with the 3D image matrix. Luckily, we can also read the raw pixel values into a NumPy array. For this, we use the GetArrayFromImage method.

```
image_array = sitk.GetArrayFromImage(image)
```

This is where things become a little bit tricky. If you print the shape of the image_array array, you will see that it doesn't match that of the SimpleITK Image.

```
print(image_array.shape)
```

While (Simple)ITK orders images as (x, y, z), NumPy orders arrays as (z, y, x). This is something to keep in mind and you can do two things:

- Just accept it and index your NumPy arrays in order (z, y, x).
- After you get your array, swap the x and y axes as follows

```
image_array = np.swapaxes(image_array, 0, 2)
```

Exercise

What is the new shape of `image_array`?

3.2.1 Physical voxel size

The image that you have loaded has some peculiarities. It is a CT image of the heart which has been acquired with (relatively) thick slices.

Exercise

Determine - based on the functions that you've seen here - what the ratio is between the slice spacing and the in-plane resolution?

Exercise

What is the volume (in ml) of a single voxel in the image?

3.3 Visualizing an image

So far, we have only considered the medical image as a 3D matrix, but we would also like to visualize it. Neither SimpleITK or NumPy provides functions to do this because that's not their purpose. We can, however, use Matplotlib to visualize the NumPy array that we now have. Matplotlib provides a wide range of functionality and it will take you a long time to fully appreciate everything it offers. For now, we will look at some key functions that will help you in this course.

The code below plots a sine wave on 100 values between 0 and 20.

```
import matplotlib.pyplot as plt

fig = plt.figure()
x = np.linspace(0, 20, 100)
y = np.sin(x)
plt.plot(x, y)
plt.show()
```

Matplotlib has many functions to display data. For images as 2D matrices, it doesn't use the plot function, but the imshow function. If we visualize the 21st slice in our 64-slice CT image, we get the following image.

```
plt.figure()
plt.imshow(image_array[:, :, 20])
plt.show()
```

Now we see what's actually in the image. It's a CT scan of a heart, where we see (on the left) the chest wall, in the center the heart, and on the right the spine. Surrounding the heart is lung tissue. However, if you would show the image like this to a radiologist, they would be confused. First of all, they are used to seeing CT images in grayscale. We can fix that by changing the Matplotlib colormap to gray.

```
plt.figure()
plt.imshow(image_array[:, :, 20], cmap = 'gray')
plt.show()
```

A second problem is that the image is rotated 90 degrees counterclockwise. This is because when talking about images, we typically use different axes than when talking about plots. For example, coordinate (20, 10) in an image means 21st row (remember we start counting at 0), 11th column. In a plot, it would mean 21st *column*, 11th *row*. So to fix that, we need to swap axes in our visualization. Because the image slice is a 2D matrix, we can just use transpose

```
plt.figure()
plt.imshow(image_array[:, :, 20].transpose(), cmap='gray')
plt.show()
```

That's more like it! Now, you may notice that the image does not show a lot of contrast. First of all, let's see which image values are present in the image.

```
plt.figure()
plt.hist(image_array[:, :, 20].flatten(), bins=50)
```

(continues on next page)

(continued from previous page)

```
plt.xlabel('Hounsfield unit')
plt.ylabel('Count')
plt.show()
```

3.3.1 Image intensities

The histogram shows a wide range of image intensities (Hounsfield units) in the image. For visualization, you might only be interested in values between 0 and 100.There are several ways to focus on different parts of the image.

- You can change the values in the image using something like image_array = np.clip(image_array, 0, 100)
- You can use the vmin and vmax options to limit the range of visible values. The latter option is preferable as it doesn't change the data.

```
plt.figure()
plt.imshow(image_array[:, :, 20].transpose(), cmap='gray', vmin=0, vmax=100)
plt.show()
```

Exercise

A reasonable range for non-contrast cardiac CT images is between -300 and 450. Visualize the image with these limits.

Exercise

The image that we have just visualized is an *axial* image slice. Can you also visualize a *sagittal* and a *coronal* image slice of the same 3D volume? Use the aspect option in Matplotlib to correct for the anisotropic pixel size.

3.4 MeVisLab

This is the end of this first Jupyter notebook. Notebooks are great for interactive Python programming, but not for medical image visualization. We have put a tutorial on Canvas to get started with MeVisLab. Follow this tutorial to visualize 3D medical images in an interactive way!

CHAPTER

FOUR

TUTORIAL 2

4.1 February 13, 2025

CHAPTER

FIVE

CONVOLUTION AND CROSS-CORRELATION

In the previous tutorial, you have looked at images. Now, we're going to look at the effect of applying some filters to our image. First, load the image that you used last week. The images used in the different tutorials are available here. For this tutorial you only need to download the folder "Tutorial 1".

```
data_path = r''
```

```
import os
import SimpleITK as sitk
import numpy as np
import matplotlib.pyplot as plt

image_file = os.path.join(data_path, r'TEV1P1CTI.mhd')

if not os.path.exists(image_file):
    print(f'The image could not be found. '
          f'Please check that the file TEV1P1CTI.mhd can be found in {data_path}')
else:
    image = sitk.ReadImage(image_file)
    image_array = sitk.GetArrayFromImage(image)
    image_array = np.swapaxes(image_array, 0, 2)
```

To make life simple, we just consider a 2D image, and we downsample the image by a factor 4 so the effect of filters is more clear. There are many ways to do this, here we use the SciPy package.

```
import scipy.ndimage as scnd
from IPython.display import display, clear_output

# Only consider one image slice
image_slice = image_array[:, :, 20].squeeze().transpose()

# Use Scipy to downsample the image by a factor 4
image_slice = scnd.zoom(image_slice, 0.25)

plt.title("Downsampled slice")
plt.imshow(image_slice, cmap="gray")
plt.show()
```

5.1 Correlation and convolution

First, we're going to visualize what the difference between convolution and cross-correlation is. You (hopefully) remember from the lecture that convolution of an image h with a kernel g is $f(i,j) = \sum_m \sum_n h(i-m, j-n)g(m,n)$ whereas cross-correlation is defined as $f(i,j) = \sum_m \sum_n h(i+m, j+n)g(m,n)$. First define a *filter* or *kernel*:

```
g = np.array([[0, 0, 0],
              [0, 0, 1],
              [0, 0, 0]])
```

Exercise

Without running the code below, try to figure out what the kernel will do when applied to an image in either convolution or cross-correlation.

Your answer goes here.

Let's first apply the kernel with cross-correlation `scnd.correlate` and see what it does. We apply the kernel a couple of times so you see what's happening.

```
filtered_image = image_slice

fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)

for level in range(20): # The kernel is applied 20 times
    filtered_image = scnd.correlate(filtered_image, g, mode='constant', cval=0)
    ax.imshow(filtered_image, cmap='gray', clim=[-300, 450])
    ax.set_title('Applied {} times'.format(level))
    display(fig)

    clear_output(wait = True)
    plt.pause(0.1)
```

Exercise

What happens when you use `scnd.correlate` instead of `scnd.convolve`. Try below. Can you explain this behavior?

```
## FILL IN ⍰
```

Exercise

Of course we're not limited to shifting images left or right, we can also define kernels to, e.g., smooth or sharpen or image. A well-known smoothing kernel is the Gaussian kernel, which resembles a 2D normal distribution. Try to implement the kernel below and apply it once to your image.

```
## FILL IN ⍰
```

Exercise

What happens if you use correlate, and what happens if you use convolve?

Your answer goes here

5.2 PyTorch

PyTorch is (along with TensorFlow) one of the two most commonly used Python libraries for deep learning and linear algebra on GPUs. Before we jump into any real deep learning, there is a couple of things that you should know. Like NumPy, PyTorch operates on matrices. However, while NumPy calls these *arrays*, PyTorch calls them *tensors*. We can easily go back and forth between NumPy arrays and PyTorch tensors. See for yourself:

```
import torch

# Start with a NumPy array
a = np.array([1, 2, 3])
print(type(a))
print(a)

# Convert to PyTorch tensor
a_t = torch.from_numpy(a)
print(type(a_t))
print(a_t)

# Convert back to NumPy array
a = a_t.numpy()
print(type(a))
print(a)
```

PyTorch is supported on CPUs, GPUs, TPUs and (newer) MacBooks. You can set your device as follows.

```
if torch.cuda.is_available():
    # CUDA
    gpu = torch.device('cuda:0')
else:
    # MacBook with >M1 chip
    gpu = torch.device('mps')

try:
    a_t = a_t.to(device=gpu)
except Exception:
    print("No GPU was found on your machine."
          "Use colab or JupyterLab to access a GPU.")
```

Note that the tensor is now on the GPU, so we cannot add a tensor that's on the CPU to it. For example, in the following line we initialize a new tensor on the CPU and try to add it but we get an error.

```
summed = a_t + torch.Tensor([4, 5, 6])
```

Exercise

Try to fix this bug so you can add the tensors on the GPU.

```
## FILL IN ⍰
```

However, best practice is to initialize your tensors and objects directly on the GPU. For example, the code below will generate a random 1000×1000 matrix on the GPU.

```
rn = torch.randn(1000, 1000, device=gpu)
print(rn)
```

Now let's perform *convolution* in PyTorch! For this, we use the conv2d function. The code below is not very different from what we have done before, but now everything happens in PyTorch instead of NumPy.

```
import torch.nn.functional as F

image_slice_t = torch.from_numpy(image_slice).float()

g_t = torch.tensor([[0, 0, 0],
                    [0, 0, 1],
                    [0, 0, 0]]).float()

# We reshape the image because the conv2d function expects [batch_size, channels,␣
↪width, height] inputs and kernels. You'll see
# later what we mean with batch_size and channels.
filtered_image_t = F.conv2d(image_slice_t.reshape((1, 1, 128, 128)), g_t.reshape((1,␣
↪1, 3, 3)), padding='same').squeeze()

fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
for level in range(1, 20):
    filtered_image_t = F.conv2d(filtered_image_t.reshape((1, 1, 128, 128)), g_t.
↪reshape((1, 1, 3, 3)), padding='same').squeeze()
    ax.imshow(filtered_image_t.numpy(), cmap='gray', clim=[-300, 450])
    ax.set_title('Applied {} times'.format(level))
    display(fig)

    clear_output(wait = True)
    plt.pause(0.1)
```

Exercise

And, what do you see? A convolution or a cross-correlation? (Why) is this surprising?

Answer:

Exercise

All kernels that we have used so far are 3×3 elements, but this is an arbitrary choice. Can you implement a kernel that lets the image make a horse jump to the top right? Implement the kernel and apply with PyTorch.

```
## FILL IN ?
```

CHAPTER

SIX

LEARNING PARAMETERS FROM SYNTHETIC DATA

In this part of the tutorial, we will define a very simple optimization problem and how to tackle it with PyTorch. In the next part of the tutorial, we will use MONAI, which is a PyTorch extension for medical images.

Like in the lecture, we will solve a linear regression problem with one independent input variable x and one dependent output variable y. The output variable is a function of the input, i.e.

$$y = wx + b$$

The measurements contain some noise, and thus, we're looking at an optimization problem

$$\underset{w,b}{arg\,min} \frac{1}{m} \sum_{i}^{m} (wx_i + b - y_i)^2$$

In this example, we will perform basic linear regression using PyTorch. First, we will generate our 'dataset', consisting of x and y values sampled along a line (with some added noise). We here use PyTorch (torch) functions to initialize our data.

```
# Generation of a data set including N samples
N = 50
x = torch.linspace(0, 4, N, device=gpu)
b = 4
w = 2
noise = torch.normal(torch.zeros(N)).to(gpu)
y = (w * x + b + noise).float()

# Plot the synthetic data set
plt.figure()
plt.title("Synthetic data")
plt.scatter(x.cpu(), y.cpu(), color='r') # We use this code to map the data back to␣
↪the CPU
plt.xlim(-1, 5)
plt.xlabel('$x$')
plt.ylim(0, 15)
plt.ylabel('$y$')
plt.show()
```

Here we want to learn a linear regression, so we don't need a deep neural network with non-linear activations, but only one fully-connected layer. In PyTorch, this is called a Linear layer, as it learns a linear relation between inputs and outputs. Take a look at the documentation to see which arguments this layer has.

The arguments we define are the following:

1. The **number of input features**, which is in this case 1, i.e., x.
2. The **number of output features**, which is in this case also 1, i.e., y.

3. [Optional] We choose to define whether or not the layer has a **bias term**. By default, this is set to True.
4. [Optional] We define here which **device** is used to instantiate the layer. In our case, we use a GPU.

```
model = torch.nn.Linear(in_features=1, out_features=1, bias=True, device=gpu)
```

Like any neural network in PyTorch, training this network requires an loss function. Because this is a regression problem, we use a mean squared error loss, or MSELoss in PyTorch, with default settings.

```
loss_func = torch.nn.MSELoss()
```

We now define a simple training procedure to fit a model to a dataset. There are a few *very* important steps in this piece of code.

- First, we create the optimizer, which is an essential part of training the model. In this case, we use an SGD optimizer, which is the gradient descent optimizer that you saw in the lecture. The optimizer keeps track of the parameters of the model and is initialized with a learning rate.
- Second, we loop over a number of iterations and in each iteration we perform the following steps
  - `optimizer.zero_grad()` resets all gradients of the model parameters. If we wouldn't do this, gradients would be added up over iterations.
  - `output = model(x.unsqueeze(1))` applies the network to the input sample.
  - `loss = loss_function(output, y.unsqueeze(1))` computes the MSE loss between the output of the network and the target, i.e., y.
  - `loss.backward()` backpropagates the prediction loss to all parameters.
  - `optimizer.step()` applied gradient descent the parameters by the gradients collected in the backward pass.

These few lines are at the core of anything you'll do in PyTorch. Note that this is only a very short description of the process, if you want to know more, take a look at this tutorial.

```
import tqdm

def train(model, iterations, lr, loss_function, x, y):
    model.reset_parameters()
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    train_loss = []
    for iteration in tqdm.tqdm(range(iterations)):
        optimizer.zero_grad()
        output = model(x.unsqueeze(1))
        loss = loss_function(output, y.unsqueeze(1))
        train_loss.append(loss.item())
        loss.backward()
        optimizer.step()
    return model, train_loss
```

Exercise

Does this piece of code use batch gradient descent, mini-batch gradient descent or stochastic gradient descent?

Answer:

Exercise

Train the model for 1000 iterations with a learning rate of 0.001 and visualize the loss over time. What happens to the loss curve if you increase or decrease the learning rate (i.e. in steps of factor 10)?

```
# FILL IN THE RIGHT ARGUMENTS ⍰
model_trained, loss = train(model=model, iterations=XXXXX, lr=XXXX, loss_
↪function=loss_func, x=x, y=y)

plt.plot(loss)
plt.xlabel('Iterations')
plt.ylabel('MSE loss')
plt.show()
```

```
# Using the code below, you can visualize the fitted line.

with torch.no_grad():
    output = model_trained(x.unsqueeze(1))
plt.plot(x.cpu(), output.cpu(), label='prediction')
plt.scatter(x.cpu(), y.cpu(), color='r', label='training data')
plt.xlim(-1, 5)
plt.xlabel('$x$')
plt.ylim(0, 15)
plt.ylabel('$y$')
plt.legend()
plt.show()
```

Exercise

Remember how when we defined `model` we set `bias` to `True`? Try to run the code again but without bias. What do you notice? The following line counts the number of trainable parameters in a model. What's the difference between the model with and without bias?

```
# This block counts the number of parameters in the model.
sum(p.numel() for p in model.parameters() if p.requires_grad)
```

```
# RETRAIN A MODEL WITHOUT BIAS ⍰
```

CHAPTER

SEVEN

A CONVOLUTIONAL NEURAL NETWORK

Medical Open Network for Artificial Intelligence (MONAI) is the library we will use in this course for deep learning on medical images. MONAI is built on the same concepts as PyTorch and its classes in many cases inherit from PyTorch classes. However, MONAI has specific functions and models that make it appropriate for medical images. The 'raw' loaded medical images are often not directly suited for training a neural network; they need to be processed before being used. MONAI offers a streamlined framework for transforming the data and feeding it into the network. In practice, you will use a combination of the two. A typical workflow in both PyTorch and MONAI is as follows:

1. Define a dataset
2. Define transforms on the dataset to make it suitable for the network
3. Build a dataloader
4. Build the network / define the optimizer, loss function and hyperparameters
5. Start training procedure

In this tutorial, we build a classification network for tumor classification, using datasets from MedMNIST. This is a collection of datasets with binary (yes or no) or multiclass labels in 2D or 3D. Each sample in these datasets is a 28×28 pixel (2D) or $28 \times 28 \times 28$ voxel (3D) image. To be able to use MONAI and this repository, we install the corresponding Python package using the following cell.

```
!pip install monai
!pip install medmnist
```

After you've installed the `medmnist` package, import it and download the `train` segment of the pneumonia dataset. This is a binary task where patients either have pneumonia or don't. Note that there is also a `validate` and `test` segment that we will download later.

```
import medmnist
dataset = medmnist.PneumoniaMNIST(split="train", download=True)
```

7.1 Datasets

The first step in the procedure consists of building a dataset that contains the images and labels. A dataset is an object, that we build using the PyTorch Dataset class template. Remember the Car class from last tutorial, with class-specific functions accelerate and brake? Here, we build a custom dataset object, where we need to define three functions in the dataset:

- `__init__`: runs when constructing the object, place where you can construct paths to the data and save them in an array, or load the images in an array.
- `__len__`: returns length of the dataset, i.e. number of samples.
- `__getitem__`: returns a sample from the dataset. This function takes an index as input, and returns the corresponding sample in the dataset. In our case, this would be two objects: an image and its corresponding label. This function returns a dictionary containing image and label. The advantage of using a dictionary is that you can keep track of transforms on the data later on in the pipeline.

The `MedMNISTData` class below inherits from the MONAI `Dataset` class, which in turn inherits from the PyTorch `Dataset` class.

```
import os
import monai

class MedMNISTData(monai.data.Dataset):

    def __init__(self, datafile, transform=None):
        self.data = datafile
        self.transform = transform


    def __getitem__(self, index):
        # Make getitem return a dictionary with keys ['img', 'label'] for the image␣
↪and label respectively
        image = self.data[index][0]
        if self.transform:
            image = self.transform(image)
        return {'img': image, 'label': self.data[index][1]}

    def __len__(self):
        return len(self.data)
```

Note that in `__getitem__` we apply a transform to each image that is selected. Such a transform is simply a function or a composition of functions that can be applied to a sample to transform it into another sample. This typically includes things like intensity normalization, cropping, resampling, etc. and are essential to most deep learning pipelines in medical imaging. Here we compose two simple transforms:

- The `ToTensor` transform converts an image into a PyTorch `Tensor` type
- The `Normalize` transform normalizes the intensities of an image to mean=0.5 and std=0.5 (in this case)

MONAI and other Python libraries like TorchIO contain many convenient functions to transform your images.

```
import torchvision.transforms as transforms

data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[.5], std=[.5])
])
```

Then, using the cell below, we can initialize a `Dataset` object based on the data that we have just downloaded.

```
train_dataset = MedMNISTData(dataset, transform=data_transform)
```

The code below visualizes the first sample from the data set.

```
plt.figure()
plt.imshow(train_dataset[0]['img'].squeeze(), cmap='gray')
plt.title('Label {}'.format(train_dataset[0]['label']))
plt.show()
```

Exercise

Visualize a number of random samples from the dataset. What is their corresponding label?

Exercise

Make a `val_dataset` and a `test_dataset` class for the validation and test set, respectively. Verify that these have 524 and 624 samples.

7.2 Dataloaders

Now that we have a dataset, we can build a dataloader that automatically generates batches with given size. As input it uses one of the data sets constructed in the previous section. Here, the batch size is 32. We shuffle all samples so that each minibatch is different.

```
train_dataloader = monai.data.DataLoader(train_dataset, batch_size=32, shuffle=True)
```

Exercise

Also initialize a dataloader for the validation set and the test set. Call these `val_dataloader` and `test_dataloader`

Exercise

With these settings, will this dataloader support batch gradient descent, mini-batch gradient descent or stochastic gradient descent?

Answer:

7.3 The network

Now our data pre-processing is all set up, we can define our neural network architecture. It is conventional to code neural networks as a torch.nn.Module object, where the init function defines the model's layers, and the forward function defines how data is fed through the network. Note that this network is built up from convolution layers, pooling layers, and the (fully connected) linear layers that we just used in the PyTorch section.

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,␣
 ↪stride=1)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3,␣
 ↪stride=1)
        self.fc1 = nn.Linear(in_features=9216, out_features=128)
        self.fc2 = nn.Linear(in_features=128, out_features=1)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = F.relu(x)
        output = self.fc2(x)
        return output

net = Net()
```

Exercise

How many layers does this network have? And how many weights?

Move the model to the GPU.

```
model = Net().to(gpu)
```

To train our network, we should again select an **optimizer** and a **loss** function.

- A very common option is to use the Adam optimizer, which is an adaptive version of stochastic gradient descent.
- Because we are solving a classification problem, a good loss function is binary cross-entropy. Here, we use `BCEWithLogitsLoss`, which operates directly on the output of the network (its logits) and integrates a sigmoid activation function.

```
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)
loss_function = torch.nn.BCEWithLogitsLoss()
```

7.4 Training procedure for classification

Now we have all the ingredients in place to start training our network. Easiest is to define a train function that performs the training procedure. In addition to a forward and backward pass with the training data, we want to perform a validation loop every N epochs, using unseen data.

```
from tqdm import tqdm

def train_medmnist(model, train_dataloader, val_dataloader, optimizer, epochs,␣
↪device=gpu, val_freq=1):
    train_loss = []
    val_loss = []

    for epoch in tqdm(range(epochs)):
        model.train()
        steps = 0
        epoch_loss = 0

        for batch in train_dataloader:
            optimizer.zero_grad()
            images = batch['img'].float().to(device)
            labels = batch['label'].float().to(device)
            output = model(images)
            loss = loss_function(output, labels)
            epoch_loss += loss.item()
            loss.backward()
            optimizer.step()
            steps += 1

        train_loss.append(epoch_loss/steps)

        # validation loop
        if epoch % val_freq == 0:
            steps = 0
            val_epoch_loss = 0
            model.eval()
            for batch in val_dataloader:
                images = batch['img'].float().to(device)
                labels = batch['label'].float().to(device)
                output = model(images)
                loss = loss_function(output, labels)
                val_epoch_loss += loss.item()
                steps += 1
            val_loss.append(val_epoch_loss/steps)

    return train_loss, val_loss, model
```

Train your (maybe?) first convolutional neural network! Time to get a coffee, this might take a few minutes.

```
val_freq = 10
n_epochs = 100
train_loss, val_loss, model = train_medmnist(model, train_dataloader, val_dataloader,␣
↪optimizer, epochs=n_epochs, val_freq=val_freq)
```

Exercise

Visualize the training and validation loss using Matplotlib. What do you see? Did your model overfit?

7.5 Part 5e: Performance evaluation

By now, you have a trained version of the CNN, that should be able to distinguish between (very small) X-ray images showing pneumonia or not. For classification tasks, the performance is often measured in terms of specificity and sensitivity, as shown in the diagram below:

For this model, we will make an ROC curve to assess the performance.

```
from sklearn.metrics import roc_curve, auc
```

```
def validation_results_ROC(model, dataloader):
    for data in dataloader:
        images = data['img'].float().to(gpu)
        labels = data['label']
        with torch.no_grad():
            output = model(images).cpu()
        fpr, tpr, _ = roc_curve(labels, output.squeeze(1))
        print(auc(fpr, tpr))
        plt.plot(fpr,tpr)
        plt.xlabel('False positive rate')
        plt.ylabel('False negative rate')
        plt.plot([0,1], [0,1], 'r--')
```

Exercise

Does the model perform well for pneumonia classification? Calculate the area under the curve for the validation set as well as for the test set.

For improvement of the model, it is useful to visualize some inputs and the corresponding predictions of the network.

```
def validation_results_visualize(model, dataset):
    index = np.random.randint(0, len(dataset))
    image = dataset[index]['img']
    plt.imshow(image.numpy().squeeze(), cmap='gray')
    image = image.float().to(gpu)
    label = dataset[index]['label'].item()
    with torch.no_grad():
        output = F.sigmoid(model(image.unsqueeze(0)))
    plt.title(f'Ground truth: {label}, prediction: {int(output)}')
```

```
validation_results_visualize(model, val_dataset)
```

Exercise

Select a 3D dataset from MedMNIST (NoduleMNIST3D or VesselMNIST3D) and train and validate a model for that set. Consider which changes you have to make.

CHAPTER

EIGHT

TUTORIAL 3

8.1 February 27, 2025

In the last two tutorials, you've taken your first steps in PyTorch and trained your first (convolutional) neural networks. You know now what convolution and cross-correlation do and what the essential steps in training a network are.

This week, you will train convolutional neural networks for 2D image segmentation. Keep in mind that everything you do and learn here can also be easily applied to 3D images. The data that we will be using today consists of chest X-ray images. The end goal in this dataset is to segment the ribs in these images, as in the image below. However, instead of segmenting and labeling all individual ribs, we will start a bit simpler and just segment every pixel that is a rib.

First, we set the data path. You can get the required images by downloading `ribs.zip` from the folder "Tutorial 3" here and uploading the folder to your coding environment.

Direct download

You can also directly download the file to your programming environment. If you are using the UT Jupyter server (as most of you are), open a terminal window on the server and run

`wget -O ribs.zip https://surfdrive.surf.nl/files/index.php/s/Y4psc2pQnfkJuoT/download`. This will download the file `ribs.zip`. Then, unzip this file by running `unzip ribs.zip` and you should find all image files on the server.

Set the data path to the "ribs" folder in the package that you have just downloaded in the code block beneath. If you're running this on Colab, don't forget to set a GPU runtime!

```
# ⚠ add path:
data_path = "../data/Tutorial_3/ribs"
```

```
# check if data_path exists:
import os

if not os.path.exists(data_path):
    print("Please update your data path to an existing folder.")
elif not set(["train", "val", "test"]).issubset(set(os.listdir(data_path))):
    print("Please update your data path to the correct folder (should contain train,␣
↪val and test folders).")
else:
    print("Congrats! You selected the correct folder :)")
```

```
# Install additional packages required for this tutorial
!pip install scikit-image
!pip install wandb
!pip install monai
```

CHAPTER

NINE

WEIGHTS AND BIASES

In the previous tutorial, you have trained a neural network and monitored the loss curves in a Jupyter notebook. You can imagine that if you train multiple networks with different settings, it's easy to lose track of all loss curves and figure out which model is best. Luckily, there exist excellent so-called MLOps systems to keep track of your experiments. Examples are Tensorboard, Neptune, and Weights & Biases (wandb). Here, we will give you some tips on how to use this last one to keep track of your experiments. Setting everything up is a simple process:

1. Register an account at https://wandb.ai/site. Your projects and runs will be shown on this website, which is accessible from anywhere.
2. Run the cell below (including `wandb.login()`) to sign into your account. You will have to paste an API key, which you can find at https://wandb.ai/authorize, and hit Enter.

```
import wandb
wandb.login()
```

Now, we can initialize a run in wandb with `wandb.init()`. You can specify the project and run name, as well as configuration settings, by passing these as additional arguments.

```
run = wandb.init(project='Example project', name='Example run', config={'dataset':
↪'VinDr-RibCXR'})
```

By running the code above, a new run is created. Although we have not logged any information yet, it is possible to see the run on the wandb website. As soon as we will start logging information, it will become visible. It is possible to log information to your run with `wandb.log()`. When it's time to complete the run, you can stop it with `run.finish()`. Run the cell below and see what happens on the run webpage.

```
import random
import time

# time.sleep(1) makes the script wait for 1 second, so the full script takes about 60␣
↪seconds to finish
for step in range(60):
    wandb.log({'Loss': random.random(), 'Accuracy': random.random()})
    time.sleep(1)

run.finish()
```

In the remainder of this notebook, we will use wandb to log our loss values to a run webpage. In this way, it is easier to keep track of multiple losses and to compare several runs.

CHAPTER

TEN

SEGMENTATION OF CHEST X-RAY IMAGES

In this part of the tutorial, you will train a convolutional neural network for automatic segmentation of ribs in a chest X-ray image. You will examine the effects of using different loss functions, data augmentations and network architectures. To keep track of all these parameters, we will use wandb.

10.1 Data management

10.1.1 Build a CacheDataset

At the beginning of the notebook, you defined the path to the rib data. We will now use it to find and load the corresponding images. The data consists of an image and a label. However, the label is in this case not a binary label, but a segmentation mask. Both the segmentation masks and the X-rays are .png files and can be opened using the PIL library.

For this tutorial we are not going to code the Dataset class ourselves but instead use a Monai class: `CacheDataset`. It has two main advantages:

- as the name suggests, it uses a cache mechanism, which means that it is memory efficient (this is partly explained later).
- everything is pre-built, so no need to code the `__init__`, `__getitem__` and `__len__` functions anymore.

`CacheDataset` only needs the list of your data samples to work. As we are learning a segmentation task, one data sample is composed of two images: the chest X-ray (named `img`) and the mask of the ribs (named `mask`). With Python, we represent this data sample with a dictionary:

Then we could give this kind of list to `CacheDataset` to create our data set:

However, as this list contains all the images, it means that all the images are always in Python memory even though they are not currently used so this is very inefficient. This is why we don't directly give the images to `CacheDataset`, but the paths to find them on our disk. Then we also need to give a `Transform` object, `LoadRibData` which will transform these paths into the corresponding images only when they are needed.

As we don't want to write this list of paths manually, we write the function `build_dict_ribs` which automatically computes it based on the root folder of the data directory `data_path`. This is possible because fortunately your teachers pre-organized the dataset for this tutorial, but be aware that most medical datasets require a lot of cleaning up prior to this.

```
import os
import numpy as np
import matplotlib.pyplot as plt
import glob
import monai
from PIL import Image
```

(continues on next page)

(continued from previous page)

```
import torch

def build_dict_ribs(data_path, mode='train'):
    """
    This function returns a list of dictionaries, each dictionary containing the keys
↪'img' and 'mask'
    that returns the path to the corresponding image.

    Args:
        data_path (str): path to the root folder of the data set.
        mode (str): subset used. Must correspond to 'train', 'val' or 'test'.

    Returns:
        (List[Dict[str, str]]) list of the dictionaries containing the paths of X-ray␣
↪images and masks.
    """
    # test if mode is correct
    if mode not in ["train", "val", "test"]:
        raise ValueError(f"Please choose a mode in ['train', 'val', 'test']. Current␣
↪mode is {mode}.")

    # define empty dictionary
    dicts = []
    # list all .png files in directory, including the path
    paths_xray = glob.glob(os.path.join(data_path, mode, 'img', '*.png'))
    # make a corresponding list for all the mask files
    for xray_path in paths_xray:
        if mode == 'test':
            suffix = 'val'
        else:
            suffix = mode
        # find the binary mask that belongs to the original image, based on indexing␣
↪in the filename
        image_index = os.path.split(xray_path)[1].split('_')[-1].split('.')[0]
        # define path to mask file based on this index and add to list of mask paths
        mask_path = os.path.join(data_path, mode, 'mask', f'VinDr_RibCXR_{suffix}_
↪{image_index}.png')
        if os.path.exists(mask_path):
            dicts.append({'img': xray_path, 'mask': mask_path})
    return dicts

class LoadRibData(monai.transforms.Transform):
    """
    This custom Monai transform loads the data from the rib segmentation dataset.
    Defining a custom transform is simple; just overwrite the __init__ function and __
↪call__ function.
    """
    def __init__(self, keys=None):
        pass

    def __call__(self, sample):
        image = Image.open(sample['img']).convert('L') # import as grayscale image
        image = np.array(image, dtype=np.uint8)
        mask = Image.open(sample['mask']).convert('L') # import as grayscale image
        mask = np.array(mask, dtype=np.uint8)
        # mask has value 255 on rib pixels. Convert to binary array
```

(continues on next page)

(continued from previous page)

```
        mask[np.where(mask==255)] = 1
        return {'img': image, 'mask': mask, 'img_meta_dict': {'affine': np.eye(2)},
                'mask_meta_dict': {'affine': np.eye(2)}}
```

Note

Note that `LoadRibData` is now outputing a more complex dictionary with two additional keys: `img_meta_dict` and `mask_meta_dict`. These contain respectively the meta data of `img` and `mask`: here we decided to add the resolution of the image, corresponding to the key `affine`. This will be useful for Monai to perform some transformations later.

A `CacheDataset` needs two arguments to perform our procedure

1. The list of samples (represented by dictionaries) which is computed by `build_dict_ribs`.
2. A transform object, `LoadRibData`, which will transform the paths in the previous list into images.

```
# construct list of dictionaries
train_dict_list = build_dict_ribs(data_path, mode='train')
# construct CacheDataset from list of paths + transform
train_dataset = monai.data.CacheDataset(train_dict_list, transform=LoadRibData())
```

Exercise

How many samples are in the training, validation, and test set?

```
# ? print the length of each subset here
```

Exercise

What are the dimensions of the images? Are they all equal? Will this a problem when training a CNN for segmentation?

```
# ? print the dimensions of a bunch of images and masks
```

Exercise

What is the pixel intensity range of the images? Are they similar? Will this be a problem when training a CNN for segmentation?

Visualize the pixel intensity range of a bunch of images. **Hint** Use Matplotlib `hist` (documentation) and `.flatten()` your image (or you will get >2000 histograms per image)

```
# Your code
```

10.1.2 Image visualization

We now build `visualize_rib_sample` to plot X-ray images on which we overlay their binary mask (in green).

```
def visualize_rib_sample(sample, title=None):
    # Visualize the x-ray and overlay the mask, using the dictionary as input
    image = np.squeeze(sample['img'])
    mask = np.squeeze(sample['mask'])
    plt.figure(figsize=[10,7])
    plt.imshow(image, 'gray')
    overlay_mask = np.ma.masked_where(mask == 0, mask == 1)
    plt.imshow(overlay_mask, 'Greens', alpha = 0.7, clim=[0,1], interpolation='nearest
↪')
    if title is not None:
        plt.title(title)
    plt.show()
```

Exercise

Use `visualize_rib_sample` to visualize samples from the training set.

```
# ❓ visualize samples here.
```

10.1.3 Data transforms

The training dataset that you have created in the previous step is quite small. In order to improve the generalization of the network, we are going to do some **data augmentation**. This can be easily implemented in the pipeline using the transforms in Monai.

Data augmentation

Data augmentation is a very common way to enlarge the number of training samples that you use. By applying transformations such as rotation, flipping, scaling to images we *artificially* enlarge the number of samples that the model sees.

Monai distinguishes *deterministic* and *random* transforms:

- *Deterministic* transforms do the exact same thing every time they are called, for example intensity normalization will always have the same outcome.
- *Random* transforms can be rotations or flips that do not have the same outcome each time they are called.

The random transforms (for example flipping and rotation) need to be performed in the same way on the input image `'img'` as well as on the label `'mask'`. Monai has adapted transforms that take dictionaries as an input, such that random transforms can be performed in the same way for both images: you don't want a different angle of rotation for the image and the mask, as they will become misaligned! You can recognize these transforms as they end with a 'd', e.g. `Zoomd` instead of `Zoom`.

The examples below show how to use one of these dictionary transforms to perform a random horizontal flip and a random rotation. The procedure of applying a transform from Monai consists of two steps:

1. Initialize the transform: here you build the object and pass the init variables, such as the probability or range of rotation, and in case of a dictionary transform the keys of the dictionary.
2. Apply the transform on a data sample

Lastly, most transforms expect a channel dimension for both 2D and 3D images. As we have only one channel, we should add this extra dimension using the `monai.transforms.AddChanneld` transform first.

```
# Load sample
index = np.random.choice(np.arange(len(train_dataset))) # This picks a random sample,␣
↪but you can change this value
sample_dict = train_dataset[index]
visualize_rib_sample(sample_dict, title="Original sample")

# Add channels
add_channels_transform = monai.transforms.AddChanneld(keys=['img', 'mask']) #␣
↪Initialize the transform
channels_sample_dict = add_channels_transform(sample_dict) # Apply the transform
print("Size of the image before AddChanneld transform", sample_dict["img"].shape)
print("Size of the image after AddChanneld transform", channels_sample_dict["img"].
↪shape)

# Random flip
# here we define the keys, the probability that the flip is performed and the axis to␣
↪flip over
random_flip_transform = monai.transforms.RandFlipd(keys=['img', 'mask'], prob=1,␣
↪spatial_axis=1)
# We put a probability of 1 to always flip the image for visualization purposes.
# Please DO NOT DO THAT in the rest of the notebook.
flipped_sample_dict = random_flip_transform(channels_sample_dict)
visualize_rib_sample(flipped_sample_dict, title="(Not quite randomly) flipped sample")

# Random rotation
# Here we define the keys in the dictionary that contain the data, the rotation range,
↪ but also the interpolation mode.
# The interpolation mode defines how new pixel values for the rotated image are␣
↪computed.
# Note that these differ between mask and image, as we want to keep binary labels for␣
↪the masks, and (bi)linear interpolation
# would result in scalar values between 0 and 1.
random_rotation_transform = monai.transforms.RandRotated(keys=['img', 'mask'], range_
↪x=np.pi/4, prob=1, mode=['bilinear', 'nearest'])
rotated_sample_dict = random_rotation_transform(channels_sample_dict)
visualize_rib_sample(rotated_sample_dict, title="Randomly rotated sample")
```

Note how we concatenated the `AddChannelsd` transform with the flip and rotate transform by applying them after each other. If we have only two transforms, like here, this is an acceptable workflow. However, if we have a whole bunch of transforms that we have to call during training of the network, you can imagine that the code will become very messy. Therefore, we can combine multiple transforms into a single one using `monai.transforms.Compose`.

Upon initialization, `Compose` takes a list of transforms as input and outputs a transform object that concatenates all the transforms. It can be applied in the same way as the transforms shown above.

Exercise

Use the `Compose` class of Monai to compose `AddChanneld`, `RandFlipd` and `RandRotated` in a single transform. Then apply this composition to a data set sample.

```
sample_dict = train_dataset[0]

# Create the composed transform
```

(continues on next page)

(continued from previous page)

```
# Apply this new single transform to sample_dict
```

Efficiency

As mentioned before, there is a distinction between random and deterministic transforms. This distinction is important, as their order can have a huge influence on computational efficiency. The CacheDataset class stores the outcomes of all deterministic transforms in memory, and performs the random transforms on-the-fly. Therefore, placing the deterministic transforms before the random ones in combination with caching results in a training and inference procedure will result in higher computational efficiency.

Exercise

Compose the `AddChannel`, `ScaleIntensity`, `Zoom`, `RandomFlip` and `RandSpatialCrop` transforms in a computationally efficient order and construct a new training dataset (with `CacheDataset`) using these transforms.

1. Make sure to visualize some samples to see if the transforms are working as you expect.
2. Don't forget to put `LoadRibData` as your first transform or your dataset will try to rotate strings representing the paths of your images…
3. If you identified problems in the previous sections it is time to solve them!

```
# ❓ code your answer here
```

10.1.4 Dataloader

We use mini-batch gradient descent during training, so we want to sample batches to train the network on, rather than single instances of the data. For this we use `monai.data.DataLoader`, which efficiently samples batches from the data that can be fed into the network right away. Remember that you have also used a `DataLoader` in Tutorial 2.

This PyTorch tutorial provides more information about datasets and dataloaders.

Exercise

Construct a dataloader with randomly sampled mini-batches of 16 images for the training set.

```
# ❓ code your answer here
```

10.1.5 Validation set

During training, we want to keep an eye on how the network generalizes to unseen data. It could perform very well on its training set, whereas it performs poorly on unseen data (overfitting). For this, we need to construct a validation set consisting of comparable but different samples of data than the training set. For our data, it can be constructed in a similar way as the train set:

1. Construct the dictionary of file paths
2. Define the transforms that should be applied on the validation data
3. Construct the CacheDataset using the dictionary and transform
4. Build a validation dataloader using from the CacheDataset

In contrast to the training set, we don't apply data augmentation to the validation set. This is important to keep in mind when defining transforms.

Exercise

Build a validation dataset and dataloader using these steps. Create a separate validation transform.

```
# ❓ code your answer here
```

10.2 Setting up the neural network, loss function, and optimizer

Now that we have the data loading process out of the way, we set up a U-Net, a loss function, and an optimizer. If possible, use a GPU to substantially speed up computing.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'The used device is {device}')
```

Setting up a U-Net for segmentation in MONAI is as easy as calling the `UNet` function and providing it with the number of input channels, output channels, and feature maps/channels in the intermediate layers. The following provides us with a model that is optimized during training to perform segmentation.

```
model = monai.networks.nets.UNet(
    spatial_dims=2,
    in_channels=1,
    out_channels=1,
    channels=(8, 16, 32, 64, 128),
    strides=(2, 2, 2, 2),
    num_res_units=2,
).to(device)
```

Exercise

How many levels does the U-Net have? And how many parameters does it have (use the code from last tutorial)?

```
# ❓ code your answer here
```

10.2.1 Loss function

The loss function should reflect what we want the training model to be able to do. For segmentation, a popular function to use for training the network is the Dice loss, which measures the overlap between the ground-truth and the model prediction. Monai offers a wide range of different loss functions that are also suitable for segmentation. In this tutorial, we will also assess the effects of using different loss functions on our network performance.

We show how to implement the Dice function in this example. Our network only has one output channel. In order to have all the output values between 0 and 1, so we can compute the Dice, the function first applies a logistic sigmoid. The Dice loss is computed over the full mini-batch (`batch=True`) to avoid poorly defined loss in individual batch samples. This means that, in a sense, we compute the Dice loss in a 3D stack of 2D images.

```
loss_function =  monai.losses.DiceLoss(sigmoid=True, batch=True)
```

10.2.2 Choose an optimizer

An optimizer algorithm is chosen that performs gradient descent on the network parameters to minimize the loss function. Last week you have used SGD (stochastic gradient descent). In many cases, Adam is a good default option. The optimizer operates on the parameters of the previously defined U-Net, i.e. `model`. A learning rate `lr` is provided to the optimizer, which defines how large the changes should be that are made to the network parameters in each iteration.

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

10.2.3 Setting up the training loop

In the previous tutorial, we explained how to set up a simple loop for training your network. For training this network, the loop can be set up in a similar way. Instead of keeping track of the train and validation loss on your computer, you can log everything in weights and biases.

Exercise

Set up a simple loop for training the network and log the following in weights and biases:

- Loss function
- Learning rate
- Training loss
- Validation loss
- Some validation images, including segmentation masks that display model output and the ground truth (see W&B documentation).

Below, we provide a `log_to_wandb()` function that you could use for this. Just call it once at the end of every epoch, with the right arguments.

```
from tqdm import tqdm
import wandb

run = wandb.init(
    project='tutorial3_segmentation',
    name='test',
```

(continues on next page)

(continued from previous page)

```
    config={
        'loss function': str(loss_function),
        'lr': optimizer.param_groups[0]["lr"],
        'transform': from_compose_to_list(train_transform),
        'batch_size': train_loader.batch_size,
    }
)
# Do not hesitate to enrich this list of settings to be able to correctly keep track␣
↪of your experiments!
# For example you should add information on your model...

run_id = run.id # We remember here the run ID to be able to write the evaluation␣
↪metrics

def wandb_masks(mask_output, mask_gt):
    """ Function that generates a mask dictionary in format that W&B requires """

    # Apply sigmoid to model ouput and round to nearest integer (0 or 1)
    sigmoid = torch.nn.Sigmoid()
    mask_output = sigmoid(mask_output)
    mask_output = torch.round(mask_output)

    # Transform masks to numpy arrays on CPU
    # Note: .squeeze() removes all dimensions with a size of 1 (here, it makes the␣
↪tensors 2-dimensional)
    # Note: .detach() removes a tensor from the computational graph to prevent␣
↪gradient computation for it
    mask_output = mask_output.squeeze().detach().cpu().numpy()
    mask_gt = mask_gt.squeeze().detach().cpu().numpy()

    # Create mask dictionary with class label and insert masks
    class_labels = {1: 'ribs'}
    masks = {
        'predictions': {'mask_data': mask_output, 'class_labels': class_labels},
        'ground truth': {'mask_data': mask_gt, 'class_labels': class_labels}
    }
    return masks

def log_to_wandb(epoch, train_loss, val_loss, batch_data, outputs):
    """ Function that logs ongoing training variables to W&B """

    # Create list of images that have segmentation masks for model output and ground␣
↪truth
    log_imgs = [wandb.Image(img, masks=wandb_masks(mask_output, mask_gt)) for img,␣
↪mask_output,
                mask_gt in zip(batch_data['img'], outputs, batch_data['mask'])]

    # Send epoch, losses and images to W&B
    wandb.log({'epoch': epoch, 'train_loss': train_loss, 'val_loss': val_loss,
↪'results': log_imgs})

# ⍰ WRITE YOUR TRAINING LOOP HERE

# Store the network parameters
torch.save(model.state_dict(), r'trainedUNet.pt')
run.finish()
```

CHAPTER

ELEVEN

EVALUATE THE TRAINED NETWORK

11.1 Visual inspection

We have trained the network on small patches of the image. In order to see how well it performs on the entire image, we can use the `monai.inferers.SlidingWindowInferer`. This 'slides' the network over patches of the input image. The overlap between patches can also be controlled.

Moreover, we want our final mask to have discrete values (0 or 1). Then, we need to discretize the continuous output of the network. This is why we use `Sigmoid` (clipping the output values between 0 and 1) and `AsDiscrete` (mapping the continuous distribution on the {0, 1} set).

```
def visual_evaluation(sample, model):
    """
    Allow the visual inspection of one sample by plotting the X-ray image, the ground␣
↪truth (green)
    and the segmentation map produced by the network (red).

    Args:
        sample (Dict[str, torch.Tensor]): sample composed of an X-ray ('img') and a␣
↪mask ('mask').
        model (torch.nn.Module): trained model to evaluate.
    """
    model.eval()
    inferer = monai.inferers.SlidingWindowInferer(roi_size=[256, 256])
    discrete_transform = monai.transforms.AsDiscrete(logit_thresh=0.5, threshold_
↪values=True)
    Sigmoid = torch.nn.Sigmoid()
    with torch.no_grad():
        output = discrete_transform(Sigmoid(inferer(sample['img'].to(device),␣
↪network=model).cpu())).squeeze()

    fig, ax = plt.subplots(1,3, figsize = [12, 10])
    # Plot X-ray image
    ax[0].imshow(sample["img"].squeeze(), 'gray')
    ax[1].imshow(sample["img"].squeeze(), 'gray')
    # Plot ground truth
    mask = np.squeeze(sample['mask'])
    overlay_mask = np.ma.masked_where(mask == 0, mask == 1)
    ax[1].imshow(overlay_mask, 'Greens', alpha = 0.7, clim=[0,1], interpolation=
↪'nearest')
    ax[1].set_title('Ground truth')
    # Plot output
```

(continues on next page)

(continued from previous page)

```
    overlay_output = np.ma.masked_where(output < 0.1, output >0.99)
    ax[2].imshow(sample['img'].squeeze(), 'gray')
    ax[2].imshow(overlay_output, 'Reds', alpha = 0.7, clim=[0,1])
    ax[2].set_title('Prediction')
    plt.show()
```

```
test_dict = build_dict_ribs(data_path, mode='test')
test_transform = monai.transforms.Compose([
        LoadRibData(),
        monai.transforms.AddChanneld(keys=['img', 'mask']),
        monai.transforms.ScaleIntensityd(keys=['img'],minv=0, maxv=1),
        monai.transforms.Zoomd(keys=['img', 'mask'], zoom=0.25, keep_size=False,␣
↪mode=['bilinear', 'nearest']),
    ]
)
test_set = monai.data.CacheDataset(test_dict, transform=test_transform)
test_loader = monai.data.DataLoader(test_set, batch_size=1)

for sample in test_loader:
    visual_evaluation(sample, model)
```

11.2 Compute evaluation metrics

Previously, we evaluated the quality of our segmentation on the test set by visually inspecting the quality of the maps produced by our network. Though this is an essential step when developping and debugging a network, this step is also quite subjective, and tedious when there are a lot of images. This is why we need metrics to evaluate the performance of our network! These metrics give us a quantity that represents the performance of our network and enable comparison to other methods.

We provide the function `compute_metric` to evaluate the performance of the network on the segmentation task. Similar to the transforms and loss functions, Monai contains many evaluation metrics to assess the model's performance. We show how to compute the Dice metric for the network.

```
def compute_metric(dataloader, model, metric_fn):
    """
    This function computes the average value of a metric for a data set.

    Args:
        dataloader (monai.data.DataLoader): dataloader wrapping the dataset to␣
↪evaluate.
        model (torch.nn.Module): trained model to evaluate.
        metric_fn (function): function computing the metric value from two tensors:
            - a batch of outputs,
            - the corresponding batch of ground truth masks.

    Returns:
        (float) the mean value of the metric
    """
    model.eval()
    inferer = monai.inferers.SlidingWindowInferer(roi_size=[256, 256])
    discrete_transform = monai.transforms.AsDiscrete(threshold=0.5)
    Sigmoid = torch.nn.Sigmoid()
```

(continues on next page)

(continued from previous page)

```
    mean_value = 0

    for sample in dataloader:
        with torch.no_grad():
            output = discrete_transform(Sigmoid(inferer(sample['img'].to(device),␣
↪network=model).cpu()))
        mean_value += metric_fn(output, sample["mask"])

    return (mean_value / len(dataloader)).item()
```

As we want to log everything in the corresponding run with W&B, we access to the corresponding run with `wandb.Api()`.

```
api = wandb.Api()
run = api.run(f"tutorial3_segmentation/{run_id}")
```

11.2.1 Dice

This is the same metric that the one that was used for the loss function.

Exercise

Find the appropriate function in `monai.metrics` to compute the mean Dice with `compute_metric`.

```
# ⍰ code your answer here
```

11.2.2 Hausdorff distance

The Hausdorff distance uses a function f which computes the minimal distance between the farthest point that can be found in a set Y compared to another set X. The Hausdorff takes the worst case between f(X,Y) and f(Y,X).

Exercise

Find the appropriate function in `monai.metrics` to compute the Hausdorff distance with `compute_metric`.

```
# ⍰ code your answer here
```

Exercise

Are you happy with the performance of your model? Can you think of ways to improve the performance?

11.3 Comparison to baseline masks

Although you did some visual inspection of the outputs of the model, interpretation of the Dice scores and Hausdorff distances is not trivial. Therefore, comparing to a baseline model for which we know it performs poorly is a good way to get some idea of how good your model actually is.

For our dummy baseline model, instead of rib-shaped segmentations, we let the model output square segmentation maps in the rib region.

```
from skimage import measure

def make_dummy_sample(sample):
    M = sample['mask'].squeeze()
    labels = measure.label(M)
    dummy_labels = np.zeros((labels.shape[0], labels.shape[1]))
    for i in np.unique(labels):
        if i > 0:
            mask_locs = np.where(labels == i)
            limits = [np.min(mask_locs[0]), np.max(mask_locs[0]), np.min(mask_
↪locs[1]), np.max(mask_locs[1])]
            dummy_labels[limits[0]:limits[1], limits[2]:limits[3]] = 1
    return torch.tensor(dummy_labels)
```

```
sample = test_set[0]
visualize_rib_sample(sample, title = 'original mask')
visualize_rib_sample({'img': sample['img'], 'mask': make_dummy_sample(sample)}, title␣
↪= 'dummy segmentation masks')
```

```
def compute_metric_dummy(dataloader, metric_fn):
    """
    This function computes the average value of a metric for a data set using the␣
↪dummy segmentation masks

    Args:
        dataloader (monai.data.DataLoader): dataloader wrapping the dataset to␣
↪evaluate.
        metric_fn (function): function computing the metric value from two tensors:
            - a batch of outputs,
            - the corresponding batch of ground truth masks.

    Returns:
        (float) the mean value of the metric
    """

    mean_value = 0
    A = monai.transforms.AddChannel()

    for sample in dataloader:
        output = make_dummy_sample(sample)
        mean_value += metric_fn(A(A(output)), sample["mask"])

    return (mean_value / len(dataloader)).item()
```

```
print(f'Mean Dice: {compute_metric_dummy(test_loader, monai.metrics.compute_
↪meandice):.3f}')
```

Exercise

Are you still happy with the performance of your model?

CHAPTER

TWELVE

PART 3: MULTICLASS CLASSIFICATION

Exercise

In the data directory, we have also placed a folder called `mask_mc` for the train, val and test set. In these images, we have labeled the left and right 7th rib. This is one of the ribs that's most prone to fracture, and identifying exactly that rib and the side that it's on could be valuable. Thus, this is no longer a binary label segmentation problem, but a multiclass segmentation problem. Try to also segment these images, consider what you will have to change in your code for this to work.

```
# ✍ code your answer here
```

CHAPTER

THIRTEEN

TUTORIAL 4

13.1 March 11, 2025

In the first tutorials, you have trained and evaluated supervised models for image classification and segmentation. In this tutorial, we will take a closer look at generative models. We will again build on PyTorch, MONAI and Weights & Biases. Let's first make sure that you have access to MONAI. As always, if you're running this on Colab, make sure that you select a runtime with GPU.

```
import torch
import random

# Check whether we're using a GPU
if torch.cuda.is_available():
    n_gpus = torch.cuda.device_count()  # Total number of GPUs
    gpu_idx = random.randint(0, n_gpus - 1)  # Random GPU index
    device = torch.device(f'cuda:{gpu_idx}')
    print('Using GPU: {}'.format(device))
else:
    device = torch.device('cpu')
    print('GPU not found. Using CPU.')
```

Pretrained models

You can download some pre-trained generator models for the GAN training exercises from this link, but of course it's much more interesting to train them yourself. Use

```
path = 'saved_models/CNN_GAN/Epoch_50.pt'
checkpoint = torch.load(path)

epoch = checkpoint['epoch']
generator.load_state_dict(checkpoint['generator'])
discriminator.load_state_dict(checkpoint['discriminator'])
optimizer_gen.load_state_dict(checkpoint['optimizer_gen'])
optimizer_dis.load_state_dict(checkpoint['optimizer_dis'])
```

to load the model (see also: this link).

You can monitor the GPU activity using `[watch] nvidia-smi` in a terminal. Ultimately you can even identify the ID of the jobs ran by your classmates to know who is using what.

Consider dialogue before disconnecting them from the network using illegal ways.

13.2 1D generative adversarial network (GAN)

We define a GAN consisting of two neural networks that play a game:

- The discriminator will learn to distinguish real and fake samples in z
- The generator will generate fake samples in z that the discriminator cannot discriminate

In the lecture, you have seen this simple 1D problem. We assume that there is a data set of real samples that are drawn from a normal distribution (the black dotted line below). These samples are in the sample domain and are called x. The generator network does not know anything about the distribution of the real samples in the sample domain, but will try to converge to a function that maps random noise z to samples that seem to come from the real sample distribution (the green line below). The discriminator network is the *adversary* of the generator and it tries to distinguish real samples from fake samples. It's predictions on x are shown with a blue curve in the figure.

First, let's set up the data for this toy problem. We define the mean value of the normal distribution from which **real** samples will be drawn in the sample domain x. In addition, we define the dimensionality of the normal distribution z from which noise samples to the generator will be drawn, i.e. the latent space. For now, we can assume a 1-dimensional noise distribution.

```
# Determines the distribution of the real samples N(real_mean, 1)
real_mean = 8
# Determines the dimensionality of the latent space
latent_dim = 1
```

The cell below defines the generator and discriminator networks. These are very simple networks that map scalars to scalars through a hidden layer. In this case, the networks are actually identical.

```
import torch
import torch.nn as nn

# The discriminator will directly classify the input value
class Discriminator_1D(nn.Module):

    def __init__(self):
        super(Discriminator_1D, self).__init__()
        self.layers = nn.Sequential(nn.Linear(in_features=1, out_features=32),
                                    nn.LeakyReLU(),
                                    nn.Linear(in_features=32, out_features=1))

    def forward(self, x):
        return self.layers(x)

# The generator will transform a single input value
class Generator_1D(nn.Module):

    def __init__(self):
        super(Generator_1D, self).__init__()
        self.layers = nn.Sequential(nn.Linear(in_features=1, out_features=32),
                                    nn.LeakyReLU(),
                                    nn.Linear(in_features=32, out_features=1))

    def forward(self, x):
        return self.layers(x)
```

Now, we will define the training functions for both networks. Consider what is actually happening in a GAN and how the inputs and outputs are connected. The overall objective function of our system is as follows

$$V^{(D)}(D,G) = \mathop{\mathbb{E}}_{x\sim p_{data}}[\log D(x)] + \mathop{\mathbb{E}}_{z\sim p_z}[\log(1 - D(G(z)))]$$

There are four important variables when training this GAN

- z: the noise that will be input to the generator
- $G(z)$: the output of the generator, i.e. the samples that should approximate the real samples
- $D(G(z))$: the discriminator's decision based on the fake sample
- x: real samples drawn from the real sample distribution

The generator G is trying to minimize the overall objective, and the discriminator D tries to maximize it. In other words, the discriminator aims to minimize the binary cross-entropy such that it predicts 1 for any real sample x and 0 for any fake sample $G(z)$. At the same time, the generator tries to get the discriminator to predict 1 any fake sample $G(z)$.

The cell below defines the optimizers for both networks. Note that while the networks share an objective function, they each have their own optimizer. We use the Adam optimizer in both cases, with the same settings. We use the stable `BCEWithLogitsLoss` loss function, that combines binary cross-entropy calculation with a sigmoid.

```
# Initialize both networks
discriminator = Discriminator_1D().to(device)
generator = Generator_1D().to(device)

# Configure optimizers and loss function
optimizer_dis = torch.optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.
↪999))
optimizer_gen = torch.optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.
↪999))

# We use the same loss function for both networks
loss_function = torch.nn.BCEWithLogitsLoss()
```

The code below will run the training loop. It's a bit different then you're used to, take a good look.

Exercise

In the lecture it was mentioned that the generator should be frozen when training the discriminator and vice versa. Do you see where this is happening?

Your answer goes here.

Now run the cell. This could take a few minutes, but below the cell you will periodically see a plot of the current situation. The plot shows the fake and real data distribution and the discriminators predictions (in blue).

```
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import display, clear_output


# We will store the losses here
gen_losses = []
dis_losses = []

# Training loop
n_samples = 500
iterations = 10000
```

(continues on next page)

(continued from previous page)

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)

for iteration in range(1, iterations + 1):

    # ========== Train Discriminator ==========

    for param in generator.parameters():
        param.requires_grad = False
    for param in discriminator.parameters():
        param.requires_grad = True
    discriminator.zero_grad()

    # Get a random set of input noise
    noise = torch.normal(0, 1, size=(n_samples, latent_dim), device=device)

    # Also get a sample from the 'real' distribution
    real = torch.normal(real_mean, 1, size=(n_samples, latent_dim), device=device)

    # Generate some fake samples using the generator
    fake = generator(noise)

    # Concatenate the fake and real images
    dis_input = torch.cat((real, fake))

    # Make labels for generated and real data (set labels for real samples to 1)
    dis_labels = torch.zeros((2 * n_samples, latent_dim), device=device)
    dis_labels[:n_samples] = 1

    # Train discriminator with this batch of samples
    predictions = discriminator(dis_input)
    dis_loss = loss_function(predictions, dis_labels)
    dis_loss.backward()
    optimizer_dis.step()
    dis_losses.append(dis_loss.detach().cpu().numpy())

    # ========== Train Generator ==========

    for param in generator.parameters():
        param.requires_grad = True
    for param in discriminator.parameters():
        param.requires_grad = False
    generator.zero_grad()

    # Get a random set of input noise
    noise = torch.normal(0, 1, size=(n_samples, latent_dim), device=device)

    # From the generator's perspective, the discriminator should predict ones for all␣
↪samples
    gen_labels = torch.ones((n_samples, latent_dim), device=device)

    # Train generator
    fake = generator(noise)
    predictions = discriminator(fake)
    gen_loss = loss_function(predictions, gen_labels)
    gen_loss.backward()
```

(continues on next page)

(continued from previous page)

```
    optimizer_gen.step()
    gen_losses.append(gen_loss.detach().cpu().numpy())

    # ========== Make plot ==========

    # For every 100th iteration, plot samples from real and fake distributions
    if iteration % 100 == 0:

        # Generate fake samples and predictions without gradient calculations
        with torch.no_grad():

            # Get fake and real samples, together with discriminator predictions from␣
↪a standard range of values
            noise = torch.normal(0, 1, size=(n_samples, latent_dim), device=device)
            fake = generator(noise)
            real = torch.normal(real_mean, 1, size=(n_samples, latent_dim),␣
↪device=device)
            predictions = torch.sigmoid(discriminator(torch.arange(-20, 20, 0.5,␣
↪device=device).view(80, 1)))

            # Make new plot and sleep half a second
            ax.cla()
            ax.set(xlim=(-20, 20), ylim=(0, 1))
            ax.hist((torch.squeeze(fake).cpu().numpy(), torch.squeeze(real).cpu().
↪numpy()), density=True, stacked=True, color=('g','k'))
            ax.scatter(np.arange(-20, 20, 0.5), predictions.cpu().numpy(), c='b')
            ax.set_title('Iteration {}'.format(iteration))
            ax.legend(['Discriminator', 'Fake', 'Real'])
            display(fig)
            clear_output(wait=True)
            plt.pause(0.5)
```

If all is well, the fake and real distributions should overlap nicely after training. The discriminator has essentially pushed the fake samples towards the real distribution and the generator is now able to transform the noise distribution into a distribution of 'real' samples!

Exercise

Can you explain what happened to the blue line during training? Why does it look like it does after training?

Your answer goes here.

Exercise

Try training the GAN with different input noise distributions for z, e.g. a uniform distribution. See if you can find a distribution for the real samples for which the generator fails to generate samples.

During training, we have stored the loss values for the discriminator and the generator. We can now plot these. In the neural networks that we've trained so far, we have tried to train a neural network such that the loss decreases until a minimum is reached.

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
```

(continues on next page)

(continued from previous page)

```
plt.plot(dis_losses)
plt.title('Discriminator loss')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.subplot(1, 2, 2)
plt.plot(gen_losses)
plt.title('Generator loss')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.show()
```

Exercise

The loss curves that you get now look different. Can you explain why they're not nicely dropping to zero? Can you explain the value in the loss in the discriminator based on the objective function of the discriminator? Consider that we actually let the discriminator optimize a binary cross-entropy loss.

Your answer goes here.

CHAPTER

FOURTEEN

MNIST SYNTHESIS

Although it is definitely nice that we can train two networks together to learn the distribution of a real data distribution, generating samples from a normal distribution is in itself not really interesting. Luckily, we can use the same principles to generate images. We will be synthesizing MNIST digits. To prepare for this, we first download the dataset. The code in this cell should look familiar:

- We compose several transforms into one transform
- The `torchvision` library conveniently lets you generate a `Dataset` object that already includes the MNIST digit dataset (and downloads it from the internet)
- The `DataLoader` is just what we have used before.

However, note that we do not use MONAI here, instead we use an original PyTorch `DataLoader`, which works in just the same way.

```
import torchvision
from torchvision.transforms import Compose, PILToTensor, ConvertImageDtype, Normalize
from torch.utils.data import DataLoader

# Define transform that converts PIL images into Tensors, with values between -1.0␣
↪and 1.0
transform = Compose([
    PILToTensor(),
    ConvertImageDtype(torch.float),
    Normalize(mean=0.5, std=0.5)
])

# Load the MNIST dataset
mnist_data = torchvision.datasets.MNIST(root='datasets', download=True,␣
↪transform=transform)
data_loader = DataLoader(mnist_data, batch_size=100, shuffle=True)
```

The below code gets a batch of samples from the MNIST dataset and let's you plot these. These are the 28×28 pixel images that you will be synthesizing.

```
def plotImages(images, dim=(10, 10), figsize=(10, 10), title=''):
    num_images = dim[0] * dim[1]
    images = images.reshape((num_images, 28, 28))
    fig = plt.figure(figsize=figsize)
    fig.suptitle(title, fontsize=14)
    for i in range(images.shape[0]):
        plt.subplot(dim[0], dim[1], i+1)
        plt.imshow(images[i], interpolation='nearest', cmap='gray_r')
        plt.axis('off')
```

(continues on next page)

(continued from previous page)

```
    plt.tight_layout()
    plt.show()

# Get first batch of images and plot them in a grid
images, labels = next(iter(data_loader))
plotImages(images.numpy(), title='MNIST examples')
```

Most deep learning tutorials build a discriminative model that is able to classify an image into one of the ten digit categories. In this exercise, we are going to do the inverse. Given a point in a latent space (which in our case will be a multi-dimensional Gaussian distribution), we are going to train the network to generate a realistic digit image for this point. The MNIST data set will be used as a set of real samples.

14.1 The discriminator

As you can see in the image above, we will need a generator and a discriminator network. Let's define these first using the cells below. First, we define the discriminator, which will classify images as either *real* or *fake*.

```
class Discriminator_MLP(nn.Module):

    def __init__(self):
        super(Discriminator_MLP, self).__init__()
        self.layers = nn.Sequential(nn.Linear(in_features=784, out_features=1024),
                                    nn.LeakyReLU(0.2),
                                    nn.Dropout(0.3),
                                    nn.Linear(in_features=1024, out_features=512),
                                    nn.LeakyReLU(0.2),
                                    nn.Dropout(0.3),
                                    nn.Linear(in_features=512, out_features=256),
                                    nn.LeakyReLU(0.2),
                                    nn.Linear(in_features=256, out_features=1),
                                    nn.Sigmoid())

    def forward(self, x):
        return self.layers(x)
```

Exercise

Is this a convolutional neural network? Why (not)?

Your answer goes here.

You could argue that digits are a bit more complex than samples from a Gaussian distribution, so let's set the latent space dimensionality for noise sampling a bit higher than 1. We will sample noise from a 10-dimensional distribution. It's good to realize that this is still *much* lower than the 784 dimensions (28x28) that our original MNIST images have.

```
latent_dim = 10
```

14.2 The generator

The generator is different than the discriminator. It should go from a low-dimensional noise vector to an MNIST image.

```
class Generator_MLP(nn.Module):

    def __init__(self):
        super(Generator_MLP, self).__init__()
        self.layers = nn.Sequential(nn.Linear(in_features=latent_dim, out_
↪features=256),
                                    nn.LeakyReLU(0.2),
                                    nn.Linear(in_features=256, out_features=512),
                                    nn.LeakyReLU(0.2),
                                    nn.Linear(in_features=512, out_features=1024),
                                    nn.LeakyReLU(0.2),
                                    nn.Linear(in_features=1024, out_features=784),
                                    nn.Tanh())

    def forward(self, x):
        return self.layers(x)
```

Exercise

Consider the activation functions of the output layers of the generator and discriminator networks. How are they different?

Your answer goes here.

14.3 The model

Now let's combine the generator and the discriminator. We train both using a binary cross-entropy objective. This is very similar to what we did before.

```
# Get networks
discriminator = Discriminator_MLP()
generator = Generator_MLP()

# Push networks to device
discriminator.to(device)
generator.to(device)

# Configure optimizers and loss function
optimizer_dis = torch.optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.
↪999))
optimizer_gen = torch.optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.
↪999))
loss = torch.nn.BCELoss()
```

We define some helper functions that will allow us to save and plot the models.

```
# Save model checkpoint in a folder called 'saved_models'
def saveModels(epoch, model_name):
    to_save = {
```

(continues on next page)

(continued from previous page)

```
        'epoch': epoch,
        'generator': generator.state_dict(),
        'discriminator': discriminator.state_dict(),
        'optimizer_gen': optimizer_gen.state_dict(),
        'optimizer_dis': optimizer_dis.state_dict()
    }
    directory = os.path.join('saved_models', model_name)
    if not os.path.exists(directory):
        os.makedirs(directory)
    torch.save(to_save, os.path.join(directory, 'Epoch_{}.pt'.format(epoch)))

# Plot generated images in a grid
def plotGeneratedImages(epoch, examples=100, dim=(10, 10), figsize=(10, 10)):
    with torch.no_grad():
        noise = torch.normal(0, 1, size=(examples, latent_dim), device=device)
        fake_images = generator(noise).cpu().numpy()
        fake_images = fake_images.reshape(examples, 28, 28)

        fig = plt.figure(figsize=figsize)
        fig.suptitle('Epoch {}'.format(epoch), fontsize=14)
        for i in range(fake_images.shape[0]):
            plt.subplot(dim[0], dim[1], i+1)
            plt.imshow(fake_images[i], interpolation='nearest', cmap='gray_r')
            plt.axis('off')
        plt.tight_layout()
        plt.show()
        return fig
```

Exercise

Take a look at the code, it's actually very similar to what we used in the first, 1D, GAN. See if you can find at least two differences.

If you run the code, synthesized images should be shown periodically.

Your answer goes here.

```
from tqdm import tqdm
import os

dis_losses = []
gen_losses = []

epochs = 20
batch_size = 100
fig = None

for epoch in range(1, epochs + 1):

    # Wrap dataloader into tqdm such that we can print progress while training
    with tqdm(data_loader, unit="iterations") as tqdm_iterator:
        tqdm_iterator.set_description('Epoch {}'.format(epoch))

        for i, batch in enumerate(tqdm_iterator):
```

(continues on next page)

(continued from previous page)

```
            # ========== Train Discriminator ==========

            # Freeze generator part
            for param in generator.parameters():
                param.requires_grad = False
            for param in discriminator.parameters():
                param.requires_grad = True
            discriminator.zero_grad()

            # Get a random set of input noise
            noise = torch.normal(0, 1, size=(batch_size, latent_dim), device=device)

            # Get real images and flatten the image dimensions
            real_images, _ = batch
            real_images = real_images.to(device).view(batch_size, 784)

            # Generate some fake MNIST images using the generator
            fake_images = generator(noise)

            # Concatenate the fake and real images
            dis_input = torch.cat((real_images, fake_images))

            # Labels for generated and real data
            dis_labels = torch.zeros((2 * batch_size, 1), device=device)

            # One-sided label smoothing
            dis_labels[:batch_size] = 0.9

            # Train discriminator with this batch of samples
            predictions = discriminator(dis_input)
            dis_loss = loss(predictions, dis_labels)
            dis_loss.backward()
            optimizer_dis.step()
            dis_losses.append(dis_loss.detach().cpu().numpy())

            # ========== Train Generator ==========

            # Freeze the discriminator part
            for param in generator.parameters():
                param.requires_grad = True
            for param in discriminator.parameters():
                param.requires_grad = False
            generator.zero_grad()

            # Train generator with a new batch of generated samples
            noise = torch.normal(0, 1, size=(batch_size, latent_dim), device=device)

            # From the generator's perspective, the discriminator should predict
            # ones for all samples
            gen_labels = torch.ones((batch_size, 1), device=device)

            # Train the GAN to predict ones
            fake_images = generator(noise)
            predictions = discriminator(fake_images)
            gen_loss = loss(predictions, gen_labels)
            gen_loss.backward()
```

(continues on next page)

(continued from previous page)

```
            optimizer_gen.step()
            gen_losses.append(gen_loss.detach().cpu().numpy())

    # Display generated images every 5th epoch
    if epoch % 5 == 0:
        clear_output(wait=True)
        fig = plotGeneratedImages(epoch)
        saveModels(epoch, 'MLP_GAN')
```

Exercise

If all is well, your model has synthesized images of digits. How cool is that?! Are you satisfied with the quality of these images? What could be improved?

Once again, we can plot the loss curves for the trained model.

Your answer goes here.

```
plt.figure(figsize=(10, 8))
plt.subplot(1, 2, 1)
plt.plot(dis_losses)
plt.title('Discriminator loss')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.subplot(1, 2, 2)
plt.plot(gen_losses)
plt.title('Generator loss')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.show()
```

Exercise

Inspect the loss curves for this model and explain what happens.

Your answer goes here.

14.4 A convolutional model

Thus far the discriminator and generator were both multilayer perceptrons. Now we're going to add in some convolutional layers to turn them into a deep convolutional GAN (DCGAN)-like architecture. This means that we have to redefine the generator network and a discriminator network. First, we define the discriminator, which is a pretty basic classification CNN, similar to what you used in Tutorial 2.

```
class Discriminator_CNN(nn.Module):

    def __init__(self):
        super(Discriminator_CNN, self).__init__()
        self.layers = nn.Sequential(nn.Conv2d(in_channels=1, out_channels=64, kernel_
↪size=5, stride=2, padding=2),
```

(continues on next page)

(continued from previous page)

```
                                    nn.LeakyReLU(0.2),
                                    nn.Dropout(0.3),
                                    nn.Conv2d(in_channels=64, out_channels=128,␣
↪kernel_size=5, stride=2, padding=2),
                                    nn.LeakyReLU(0.2),
                                    nn.Dropout(0.3),
                                    nn.Flatten(),
                                    nn.Linear(in_features=128*7*7, out_features=1),
                                    nn.Sigmoid())

    def forward(self, x):
        return self.layers(x)

class Generator_CNN(nn.Module):

    def __init__(self):
        super(Generator_CNN, self).__init__()
        self.linear = nn.Sequential(nn.Linear(in_features=latent_dim, out_
↪features=128*7*7),
                                    nn.LeakyReLU(0.2))
        self.convolutional = nn.Sequential(nn.Upsample(size=(14, 14)),
                                           nn.Conv2d(in_channels=128, out_channels=64,
↪ kernel_size=5, padding='same'),
                                           nn.LeakyReLU(0.2),
                                           nn.Upsample(size=(28, 28)),
                                           nn.Conv2d(in_channels=64, out_channels=1,␣
↪kernel_size=5, padding='same'),
                                           nn.Tanh())

    def forward(self, x):
        x = self.linear(x)
        x = x.view(-1, 128, 7, 7)
        x = self.convolutional(x)
        return x
```

Let's build our GAN model like before.

```
# Get networks
discriminator = Discriminator_CNN()
generator = Generator_CNN()

# Push networks to device
discriminator.to(device)
generator.to(device)

# Configure optimizers and loss function
optimizer_dis = torch.optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.
↪999))
optimizer_gen = torch.optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.
↪999))
loss = torch.nn.BCELoss()
```

Train the model using the code below. Inspect the samples that come out.

Exercise

What are some differences between these samples and the ones generated by the multilayer perceptron GAN? Can you explain these differences?

Your answer goes here.

```
dis_losses = []
gen_losses = []

epochs = 50
batch_size = 100

for epoch in range(1, epochs + 1):

    # Wrap dataloader into tqdm such that we can print progress while training
    with tqdm(data_loader, unit="iterations") as tqdm_iterator:
        tqdm_iterator.set_description('Epoch {}'.format(epoch))

        for i, batch in enumerate(tqdm_iterator):

            # ========== Train Discriminator ==========

            # Freeze generator part
            for param in generator.parameters():
                param.requires_grad = False
            for param in discriminator.parameters():
                param.requires_grad = True
            discriminator.zero_grad()

            # Get a random set of input noise
            noise = torch.normal(0, 1, size=(batch_size, latent_dim), device=device)

            # Get real images and flatten the image dimensions
            real_images, _ = batch
            real_images = real_images.to(device)

            # Generate some fake MNIST images using the generator
            fake_images = generator(noise)

            # Concatenate the fake and real images
            dis_input = torch.cat((real_images, fake_images))

            # Labels for generated and real data
            dis_labels = torch.zeros((2 * batch_size, 1), device=device)

            # One-sided label smoothing
            dis_labels[:batch_size] = 0.9

            # Train discriminator with this batch of samples
            predictions = discriminator(dis_input)
            dis_loss = loss(predictions, dis_labels)
            dis_loss.backward()
            optimizer_dis.step()
            dis_losses.append(dis_loss.detach().cpu().numpy())

            # ========== Train Generator ==========

            # Freeze the discriminator part
```

(continues on next page)

(continued from previous page)

```
            for param in generator.parameters():
                param.requires_grad = True
            for param in discriminator.parameters():
                param.requires_grad = False
            generator.zero_grad()

            # Train generator with a new batch of generated samples
            noise = torch.normal(0, 1, size=(batch_size, latent_dim), device=device)

            # From the generator's perspective, the discriminator should predict
            # ones for all samples
            gen_labels = torch.ones((batch_size, 1), device=device)

            # Train the GAN to predict ones
            fake_images = generator(noise)
            predictions = discriminator(fake_images)
            gen_loss = loss(predictions, gen_labels)
            gen_loss.backward()
            optimizer_gen.step()
            gen_losses.append(gen_loss.detach().cpu().numpy())

    # Every 5th epoch, display generated images and save model
    if epoch % 5 == 0:
        clear_output(wait=True)
        plotGeneratedImages(epoch)
        saveModels(epoch, 'CNN_GAN')
```

14.5 Interpolation in the latent space

In the past two models, we have used a 10-dimensional latent space. We're going to explore the content of this latent space a bit more. We randomly pick two points in the latent space and make a linear interpolation between these two points. Then we generate images from each of the interpolated latent points.

```
# Sample two points from noise distribution
noise_a = torch.normal(0, 1, size=(1, latent_dim), device=device)
noise_b = torch.normal(0, 1, size=(1, latent_dim), device=device)

# Interpolate in steps of 10% between the two points
noise = torch.zeros((10, latent_dim), dtype=torch.float, device=device)
for i in range(10):
    ni = i * 0.1
    noise[i, :] = ni * noise_a + (1 - ni) * noise_b

# Generate images from interpolated points
with torch.no_grad():
    fake_images = generator(noise)
    fake_images = fake_images.cpu().numpy()
    plotImages(fake_images, dim=(1, 10), figsize=(10, 2), title='Generated images␣
↪from interpolated points')
```

Exercise

Explain what you see in this figure.

Your answer goes here.

Exercise

Interpolation works, but what happens when you extrapolate out of the latent space distribution? Consider how the noise vectors are drawn. Inspect generated samples that are further away from the mode of your latent space.

```
# Your code goes here
```

CHAPTER

FIFTEEN

MEDMNIST IMAGE SYNTHESIS

In this section, we're going to synthesize images from MedMNIST. This is a collection of datasets with binary (yes or no) or multiclass labels in 2D or 3D. If you didn't do it in Tutorial 2 you have to first install `medmnist` with the following commandline:

```
!pip install medmnist
```

As before, we can define a `MedMNISTData` class for these kinds of images.

```
import os
import monai
import torchvision.transforms as transforms
import medmnist

class MedMNISTData(monai.data.Dataset):

    def __init__(self, datafile, transform=None):
        self.data = datafile
        self.transform = transform


    def __getitem__(self, index):
        # Make getitem return one tensor corresponding to the image
        image = self.data[index][0]
        if self.transform:
            image = self.transform(image)
        return image

    def __len__(self):
        return len(self.data)

data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[.5], std=[.5])
])

dataset = medmnist.PneumoniaMNIST(split="train", download=True)
train_dataset = MedMNISTData(dataset, transform=data_transform)
train_loader = DataLoader(train_dataset, batch_size=10, shuffle=True)
```

Exercise

In this second-to-last part of the practical you're going to repurpose the code that you have used so far to synthesize MedMNIST images. By now you should have sufficient experience with Python that you are able to fill in the block of

code below and train your model. Reach out if you get stuck! Good luck!

```
## YOUR CODE GOES HERE
```

15.1 Conditional MedMNIST synthesis

Exercise

For all MedMMNIST samples we already have labels (0, 1, …, n). Try to change the MedMNIST synthesis code such that you can ask the generator to generate specific labels. I.e., try to train a conditional GAN. You can look for some inspiration in this paper, in particular Sec. 4.1. Remember that you already got the MedMNIST labels when loading the data set and you used them in Tutorial 2. You will have to add these labels to the discriminator and generator network. The easiest way is to just concatenate them to your input vectors.

CHAPTER

SIXTEEN

TUTORIAL 5

16.1 March 20, 2025

In the previous tutorials, you have familiarized yourself with PyTorch, MONAI, and Weights & Biases. In last week's lecture, you have learned about registration. In this tutorial, you will develop, train, and evaluate a CNN for denoising of (synthetic) CT images.

First, let's take care of the necessities:

- If you're using Google Colab, make sure to select a GPU Runtime.
- Connect to Weights & Biases using the code below.
- Install a few libraries that we will use in this tutorial.

```
import os
import wandb

os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
wandb.login()
```

```
!pip install dival
!pip install kornia
```

16.2 Reconstruction

In this tutorial, you will reconstruct CT images. To not use too much disk storage, we will synthetise images on the fly using the Deep Inversion Validation Library (dival). These are 2D images with 128×128 pixels that contain a random number of ellipses with random sizes and random intensities.

First, make a dataset of ellipses. This will make an object that we can call for images using a generator. Next, we take a look at what this dataset contains. We will use the `generator` to ask for a sample. Each sample contains a sinogram and a ground truth (original) synthetic image that we can visualize. You may recall from the lecture that the sinogram is made up of integrals along projections. The horizontal axis in the sinogram corresponds to the location s along the detector, the vertical axis to the projection angle θ.

```
import dival
```

(continues on next page)

(continued from previous page)

```
dataset = dival.get_standard_dataset('ellipses', impl='skimage')
dat_gen = dataset.generator(part='train')
```

Run the cell below to show a sinogram and image in the dataset.

```
import numpy as np
import matplotlib.pyplot as plt

# Get a sample from the generator
sinogram, ground_truth = next(dat_gen)
fig, axs = plt.subplots(1, 2, figsize=(10, 5))

# Show the sinogram
axs[0].imshow(sinogram, cmap='gray', extent=[0, 183, -90, 90])
axs[0].set_title('Sinogram')
axs[0].set_xlabel('$s$')
axs[0].set_ylabel('$\Theta$')

# Show the ground truth image
axs[1].imshow(ground_truth, cmap='gray')
axs[1].set_title('Ground truth')
axs[1].set_xlabel('$x$')
axs[1].set_ylabel('$y$')
plt.show()
```

Exercise

What kind of CT reconstruction problem is this? Limited-view or sparse-angle CT? Why?

Answer:

Not only does the sinogram contain few angles, it also contains added white noise. If we simply backproject the sinogram to the image domain we end up with a low-quality image. Let's give it a try using the standard Filtered Backprojection (FBP) algorithm for CT and its implementation in scikit-image.

```
import skimage.transform as sktr

# Get a sample from the generator
sinogram, ground_truth = next(dat_gen)
sinogram = np.asarray(sinogram).transpose()

# This defines the projectiona angles
theta = np.linspace(-90., 90., sinogram.shape[1], endpoint=True)

# Perform FBP
fbp_recon = sktr.iradon(sinogram, theta=theta, filter_name='ramp')[28:-27, 28:-27]
fig, axs = plt.subplots(1, 3, figsize=(12, 4))
axs[0].imshow(sinogram.transpose(), cmap='gray', extent=[0, 183, -90, 90])
axs[0].set_title('Sinogram')
axs[0].set_xlabel('$s$')
axs[0].set_ylabel('$\Theta$')
axs[1].imshow(ground_truth, cmap='gray', clim=[0, 1])
axs[1].set_title('Ground truth')
axs[1].set_xlabel('$x$')
```

(continues on next page)

(continued from previous page)

```
axs[1].set_ylabel('$y$')
axs[2].imshow(fbp_recon, cmap='gray', clim=[0, 1])
axs[2].set_title('FBP')
axs[2].set_xlabel('$x$')
axs[2].set_ylabel('$y$')
plt.show()
```

Exercise

What do you think of the quality of the reconstructed FBP algorithm? Use the cell below to quantify the similarity between the images using the structural similarity index (SSIM). Does this reflect your intuition? Also compute the PSNR using the `peak_signal_noise_ratio` method in `scikit-image`.

```
import skimage.metrics as skme

print('SSIM = {:.2f}'.format(skme.structural_similarity(np.asarray(ground_truth), fbp_
↪recon, data_range=np.max(ground_truth)-np.min(ground_truth))))
# ❓ FILL IN
```

16.2.1 Datasets and dataloaders

Our (or your) goal now is to obtain high(er) quality reconstructed images based on the sinogram measurements. As you have seen in the lecture, this can be done in four ways:

1. Train a reconstruction method that directly maps from the measurement (sinogram) domain to the image domain.
2. **Preprocessing** Clean up the sinogram using a neural network, then backproject to the image domain.
3. **Postprocessing** First backproject to the image domain, then improve the reconstruction using a neural network.
4. Iterative methods that integrate data consistency.

Here, we will follow the third approach, postprocessing. We create reconstructions from the generated sinograms using filtered backprojection and use a neural network to learn corrections on this FBP image and improve the reconstruction, as shown in the image below. The data that we need for training this network is the reconstructions from FBP, and the ground-truth reconstructions from the dival dataset.

We will make a training dataset of 512 samples from the ellipses dival dataset that we store in a MONAI `DataSet`. The code below does this in four steps:

1. Create a `dival` generator that creates sinograms and ground-truth reconstructions.
2. Make a dictionary (like we did in the previous tutorial) that contains the ground-truth reconstructions and the reconstructions constructed by FBP as separate keys.
3. Define the transforms for the data (also like the previous tutorial). In this case we require an additional 'channels' dimension, as that is what the neural network expects. We will not make use of extra data augmentation.
4. Construct the dataset using the dictionary and the defined transform.

```
import tqdm
import monai

theta = np.linspace(-90., 90., sinogram.shape[1], endpoint=True)
```

(continues on next page)

(continued from previous page)

```
# Make a generator for the training part of the dataset
train_gen = dataset.generator(part='train')
train_samples = []

# Make a list of (in this case) 512 random training samples. We store the filtered␣
↪backprojection (FBP) and ground truth image
# in a dictionary for each sample, and add these to a list.
for ns in tqdm.tqdm(range(512)):
    sinogram, ground_truth = next(train_gen)
    sinogram = np.asarray(sinogram).transpose()
    fbp_recon = sktr.iradon(sinogram, theta=theta, filter_name='ramp')[28:-27, 28:-27]
    train_samples.append({'fbp': fbp_recon, 'ground_truth': np.asarray(ground_truth)})

# You can add or remove transforms here
train_transform = monai.transforms.Compose([
    monai.transforms.AddChannel(keys=['fbp', 'ground_truth'])
])

# Use the list of dictionaries and the transform to initialize a MONAI CacheDataset
train_dataset = monai.data.CacheDataset(train_samples, transform=train_transform)
```

Exercise

Also make a validation dataset and call it `val_dataset`. This dataset can be smaller, e.g., 64 or 128 samples.

```
# Your code goes here
```

Exercise

Now, make a dataloader for both the validation and training data, called `train_loader` and `validation_loader`, that we can use for sampling batches during training of the network. Give them a reasonable batch size, e.g., 16.

```
# ⍰ FILL IN
train_loader = ...
validation_loader = ...
```

16.2.2 Model

Now that we have datasets and dataloaders, the next step is to define a model, optimizer and criterion. Because we want to improve the FBP-reconstructed image, we are dealing with an image-to-image task. A standard U-Net as implemented in MONAI is therefore a good starting point. First, make sure that you are using the GPU (CUDA), otherwise training will be extremely slow.

```
import torch

if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
```

(continues on next page)

(continued from previous page)

```
else:
    device = "cpu"
print(f'The used device is {device}')
```

Exercise

Initialize a U-Net with the correct settings, e.g. channels and dimensions, and call it `model`. Here, it's convenient to use the `BasicUNet` as implemented in MONAI.

16.2.3 Loss function

An important aspect is the loss function that you will use to optimize the model. The problem that we are trying to solve using a neural network is a *regression* problem, which differs from the *classification* approach we covered in the segmentation tutorial. Instead of classifying each pixel as a certain class, we alter their intensities to obtain a better overall reconstruction of the image.

Because this task is substantially different, we need to change our loss function. In the previous tutorial we used the Dice loss, which measures the overlap for each of the classes to segment. In this case, an L2 (mean squared error) or L1 (mean average error) loss suits our objective. Alternatively, we can use a loss that aims to maximize the structural similarity (SSIM). For this, we use the kornia library.

```
import kornia

# Three loss functions, turn them on or off by commenting

loss_function = torch.nn.MSELoss()
# loss_function = torch.nn.L1Loss()
# loss_function = kornia.losses.SSIMLoss(window_size=3)
```

As in previous tutorials, we use an adaptive SGD (Adam) optimizer to train our network. This tutorial, we add a learning rate scheduler. This scheduler lowers the learning rate every *step_size* steps, meaning that the optimizer will take smaller steps in the direction of the gradient after a set amount of epochs. Therefore, the optimizer can potentially find a better local minimum for the weights of the neural network.

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=50, gamma=0.1)
```

Exercise

Complete the code below and train the U-Net.

What does the model learn? Look carefully at how we determine the output of the model. Can you describe what happens in the following line: `outputs = model(batch_data['fbp'].float().to(device)) + batch_data["fbp"].float().to(device)`?

```
from tqdm.notebook import tqdm
import wandb
from skimage.metrics import structural_similarity as ssim
```

(continues on next page)

(continued from previous page)

```
run = wandb.init(
    project='tutorial3_reconstruction',
    name='test',
    config={
        'loss function': str(loss_function),
        'lr': optimizer.param_groups[0]["lr"],
        'batch_size': train_loader.batch_size,
    }
)
# Do not hesitate to enrich this list of settings to be able to correctly keep track␣
↪of your experiments!
# For example you should include information on your model architecture

run_id = run.id # We remember here the run ID to be able to write the evaluation␣
↪metrics

def log_to_wandb(epoch, train_loss, val_loss, batch_data, outputs):
    """ Function that logs ongoing training variables to W&B """

    # Create list of images that have segmentation masks for model output and ground␣
↪truth
    # log_imgs = [wandb.Image(PIL.Image.fromarray(img.detach().cpu().numpy())) for␣
↪img in outputs]
    val_ssim = []
    for im_id in range(batch_data['ground_truth'].shape[0]):
        val_ssim.append(ssim(batch_data['ground_truth'].detach().cpu().numpy()[im_id,␣
↪0, :, :].squeeze(),
                             outputs.detach().cpu().numpy()[im_id, 0, :, :].squeeze()␣
↪))
    val_ssim = np.mean(np.asarray(val_ssim))
    # Send epoch, losses and images to W&B
    wandb.log({'epoch': epoch, 'train_loss': train_loss, 'val_loss': val_loss, 'val_
↪ssim': val_ssim})

for epoch in tqdm(range(75)):
    model.train()
    epoch_loss = 0
    step = 0
    for batch_data in train_loader:
        step += 1
        optimizer.zero_grad()
        outputs = model(batch_data["fbp"].float().to(device)) + batch_data["fbp"].
↪float().to(device)
        # FILL IN
    # validation part
    step = 0
    val_loss = 0
    for batch_data in validation_loader:
        step += 1
        model.eval()
        outputs = model(batch_data['fbp'].float().to(device)) + batch_data["fbp"].
↪float().to(device)
        # FILL IN
    log_to_wandb(epoch, train_loss, val_loss, batch_data, outputs)
    # Scheduler also needs to make a step during training
    scheduler.step()
```

(continues on next page)

(continued from previous page)

```
# Store the network parameters
torch.save(model.state_dict(), r'trainedUNet.pt')
run.finish()
```

Exercise

Now make a `DataSet` and `DataLoader` for the test set. Just a handful of images should be enough.

```
import tqdm

test_gen = dataset.generator(part='test')
....
test_dataset = ....

test_loader = monai.data.DataLoader(test_dataset, batch_size=1)
```

Exercise

Visualize a number of reconstructions from the neural network and compare them to the fbp reconstructed images, using the code below. The performance of the network is evaluated using the structural similarity function in scikit-image. Does the neural network improve this metric a lot compared to the filtered back projection?

```
model.eval()

for test_sample in test_loader:
    output = model(test_sample['fbp'].to(device)) + test_sample['fbp'].to(device)
    output = output.detach().cpu().numpy()[0, 0, :, :].squeeze()
    ground_truth = test_sample['ground_truth'][0, 0, :, :].squeeze()
    fbp_recon = test_sample['fbp'][0, 0, :, :].squeeze()
    fig, axs = plt.subplots(1, 3, figsize=(12, 4))
    axs[0].imshow(fbp_recon, cmap='gray', clim=[0, 1])
    axs[0].set_title('FBP SSIM={:.2f}'.format(ssim(ground_truth.cpu().numpy(), fbp_
↪recon.cpu().numpy())))
    axs[0].set_xlabel('$x$')
    axs[0].set_ylabel('$y$')
    axs[1].imshow(ground_truth, cmap='gray', clim=[0, 1])
    axs[1].set_title('Ground truth')
    axs[1].set_xlabel('$x$')
    axs[1].set_ylabel('$y$')
    axs[2].imshow(output, cmap='gray', clim=[0, 1])
    axs[2].set_title('CNN SSIM={:.2f}'.format(ssim(ground_truth.cpu().numpy(),␣
↪output)))
    axs[2].set_xlabel('$x$')
    axs[2].set_ylabel('$y$')
    plt.show()
```

Exercise

Instead of a U-Net, try a different model, e.g., a SegResNet in MONAI. Evaluate how the different loss functions affect the performance of the network. Notes that the SSIM on the validation set is also written to Weights & Biases during

training. Which loss leads to the best SSIM scores? Which loss results in the worst SSIM scores?

Answer:

16.3 From post-processing to pre-processing

So far, you have used a post-processing approach for reconstruction. In the lecture, we have discussed an alternative *pre-processing* approach, in which the sinogram image is improved before FBP. This additional exercise is **entirely optional**, but you could try to turn the current model into such a model, and see if the results that you get are better or worse than the results obtained so far. Good luck!

CHAPTER

SEVENTEEN

TUTORIAL 6

17.1 April 2, 2025

In this tutorial you will develop, train, and evaluate a CNN that learns to perform deformable image registration in chest X-ray images.

First, let's take care of the necessities:

- If you're using Google Colab, make sure to select a GPU Runtime.
- Connect to Weights & Biases using the code below.
- Install a few libraries that we will use in this tutorial.

```
import os
import wandb

os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
wandb.login()
```

```
!pip install monai
```

17.2 Part 1 - Registration

```
import monai
import numpy as np
import matplotlib.pyplot as plt
import torch
import wandb
```

We will register chest X-ray images. We will reuse the data of **Tutorial 3**. As always, we first set the paths. This should be the path ending in 'ribs'. If you don't have the data set anymore, you can download it using the lines below:

```
!wget https://surfdrive.surf.nl/files/index.php/s/Y4psc2pQnfkJuoT/download -O␣
↪Tutorial_3.zip
!unzip -qo Tutorial_3.zip
data_path = "ribs"
```

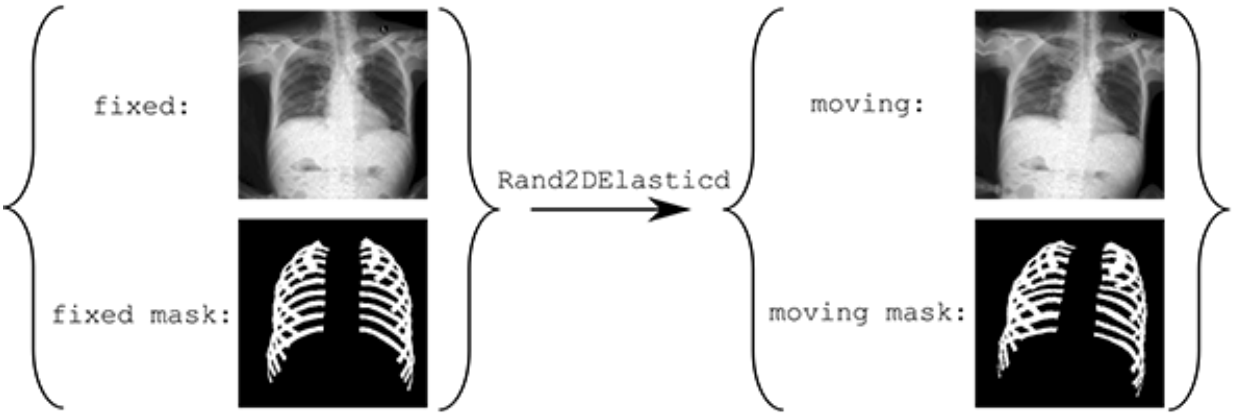
```
# ONLY IF YOU USE JUPYTER: ADD PATH
data_path = r'ribs'# WHEREDIDYOUPUTTHEDATA?
```

```
# ONLY IF YOU USE COLAB: ADD PATH
from google.colab import drive

drive.mount('/content/drive')
data_path = r'/content/drive/My Drive/Tutorial3'
```

```
# check if data_path exists:
import os

if not os.path.exists(data_path):
    print("Please update your data path to an existing folder.")
elif not set(["train", "val", "test"]).issubset(set(os.listdir(data_path))):
    print("Please update your data path to the correct folder (should contain train,␣
↪val and test folders).")
else:
    print("Congrats! You selected the correct folder :)")
```

17.2.1 Data management

In this part we prepare all the tools needed to load and visualize our samples. One thing we *could* do is perform **inter**-patient registration, i.e., register two chest X-ray images of different patients. However, this is a very challenging problem. Instead, to make our life a bit easier, we will perform **intra**-patient registration: register two images of the same patient. For each patient, we make a synthetic moving image by applying some random elastic deformations. To build this data set, we we used the Rand2DElasticd transform on both the image and the mask. We will use a neural network to learn the deformation field between the fixed image and the moving image.



Similarly as in **Tutorial 3**, make a dictionary of the image file names.

```
import os
import numpy as np
import matplotlib.pyplot as plt
import glob
import monai
from PIL import Image
import torch

def build_dict_ribs(data_path, mode='train'):
```

(continues on next page)

(continued from previous page)

```
    """
    This function returns a list of dictionaries, each dictionary containing the keys
↪'img' and 'mask'
    that returns the path to the corresponding image.

    Args:
        data_path (str): path to the root folder of the data set.
        mode (str): subset used. Must correspond to 'train', 'val' or 'test'.

    Returns:
        (List[Dict[str, str]]) list of the dictionnaries containing the paths of X-
↪ray images and masks.
    """
    # test if mode is correct
    if mode not in ["train", "val", "test"]:
        raise ValueError(f"Please choose a mode in ['train', 'val', 'test']. Current␣
↪mode is {mode}.")

    # define empty dictionary
    dicts = []
    # list all .png files in directory, including the path
    paths_xray = glob.glob(os.path.join(data_path, mode, 'img', '*.png'))
    # make a corresponding list for all the mask files
    for xray_path in paths_xray:
        if mode == 'test':
            suffix = 'val'
        else:
            suffix = mode
        # find the binary mask that belongs to the original image, based on indexing␣
↪in the filename
        image_index = os.path.split(xray_path)[1].split('_')[-1].split('.')[0]
        # define path to mask file based on this index and add to list of mask paths
        mask_path = os.path.join(data_path, mode, 'mask', f'VinDr_RibCXR_{suffix}_
↪{image_index}.png')
        if os.path.exists(mask_path):
            dicts.append({'fixed': xray_path, 'moving': xray_path, 'fixed_mask': mask_
↪path, 'moving_mask': mask_path})
    return dicts

class LoadRibData(monai.transforms.Transform):
    """
    This custom Monai transform loads the data from the rib segmentation dataset.
    Defining a custom transform is simple; just overwrite the __init__ function and __
↪call__ function.
    """
    def __init__(self, keys=None):
        pass

    def __call__(self, sample):
        fixed = Image.open(sample['fixed']).convert('L') # import as grayscale image
        fixed = np.array(fixed, dtype=np.uint8)
        moving = Image.open(sample['moving']).convert('L') # import as grayscale image
        moving = np.array(moving, dtype=np.uint8)
        fixed_mask = Image.open(sample['fixed_mask']).convert('L') # import as␣
↪grayscale image
        fixed_mask = np.array(fixed_mask, dtype=np.uint8)
```

(continues on next page)

(continued from previous page)

```
        moving_mask = Image.open(sample['moving_mask']).convert('L') # import as␣
↪grayscale image
        moving_mask = np.array(moving_mask, dtype=np.uint8)
        # mask has value 255 on rib pixels. Convert to binary array
        fixed_mask[np.where(fixed_mask==255)] = 1
        moving_mask[np.where(moving_mask==255)] = 1
        return {'fixed': fixed, 'moving': moving, 'fixed_mask': fixed_mask, 'moving_
↪mask': moving_mask, 'img_meta_dict': {'affine': np.eye(2)},
                'mask_meta_dict': {'affine': np.eye(2)}}
```

Then we make a training dataset like before. The `Rand2DElasticd` transform here determines how much deformation is in the 'moving' image.

```
train_dict_list = build_dict_ribs(data_path, mode='train')

# constructDataset from list of paths + transform
transform = monai.transforms.Compose(
[
    LoadRibData(),
    monai.transforms.EnsureChannelFirstd(keys=['fixed', 'moving', 'fixed_mask',
↪'moving_mask'], channel_dim='no_channel'),
    monai.transforms.Resized(keys=['fixed', 'moving', 'fixed_mask', 'moving_mask'],␣
↪spatial_size=(256, 256),  mode=['bilinear', 'bilinear', 'nearest', 'nearest']),
    monai.transforms.HistogramNormalized(keys=['fixed', 'moving']),
    monai.transforms.ScaleIntensityd(keys=['fixed', 'moving'], minv=0.0, maxv=1.0),
    monai.transforms.Rand2DElasticd(keys=['moving', 'moving_mask'], spacing=(64, 64),
                                    magnitude_range=(-8, 8), prob=1, mode=['bilinear',
↪ 'nearest']),
])
train_dataset = monai.data.Dataset(train_dict_list, transform=transform)
```

Exercise

Visualize fixed and moving training images associated to their comparison image with the `visualize_fmc_sample` function below.

Try different methods to create the comparison image. How well do these different methods allow you to qualitatively assess the quality of the registration?

More information on this method is available in the scikit-image documentation.

```
def visualize_fmc_sample(sample, method="checkerboard"):
    """
    Plot three images: fixed, moving and comparison.

    Args:
        sample (dict): sample of dataset created with `build_dataset`.
        method (str): method used by `skimage.util.compare_image`.
    """
    import skimage.util as skut

    skut_methods = ["diff", "blend", "checkerboard"]
    if method not in skut_methods:
        raise ValueError(f"Method must be chosen in {skut_methods}.\n"
```

(continues on next page)

(continued from previous page)

```
                        f"Current value is {method}.")


    fixed = np.squeeze(sample['fixed'])
    moving = np.squeeze(sample['moving'])
    comp_checker = skut.compare_images(fixed, moving, method=method)
    axs = plt.figure(constrained_layout=True, figsize=(15, 5)).subplot_mosaic("FMC")
    axs['F'].imshow(fixed, cmap='gray')
    axs['F'].set_title('Fixed')
    axs['M'].imshow(moving, cmap='gray')
    axs['M'].set_title('Moving')
    axs['C'].imshow(comp_checker, cmap='gray')
    axs['C'].set_title('Comparison')
    plt.show()
```

```
sample = train_dataset[0]
for method in ["diff", "blend", "checkerboard"]:
    print(f"Method {method}")
    visualize_fmc_sample(sample, method=method)
```

Now we apply a little trick. Because applying the random deformation in each training iteration will be very costly, we only apply the deformation once and we make a new dataset based on the deformed images. Running the cell below may take a few minutes.

```
import tqdm

train_loader = monai.data.DataLoader(train_dataset, batch_size=1, shuffle=False)

samples = []
for train_batch in tqdm.tqdm(train_loader):
    samples.append(train_batch)

# Make a new dataset and dataloader using the transformed images
train_dataset = monai.data.Dataset(samples, transform=monai.transforms.
 ↪SqueezeDimd(keys=['fixed', 'moving', 'fixed_mask', 'moving_mask']))
train_loader = monai.data.DataLoader(train_dataset, batch_size=16, shuffle=False)
```

Exercise

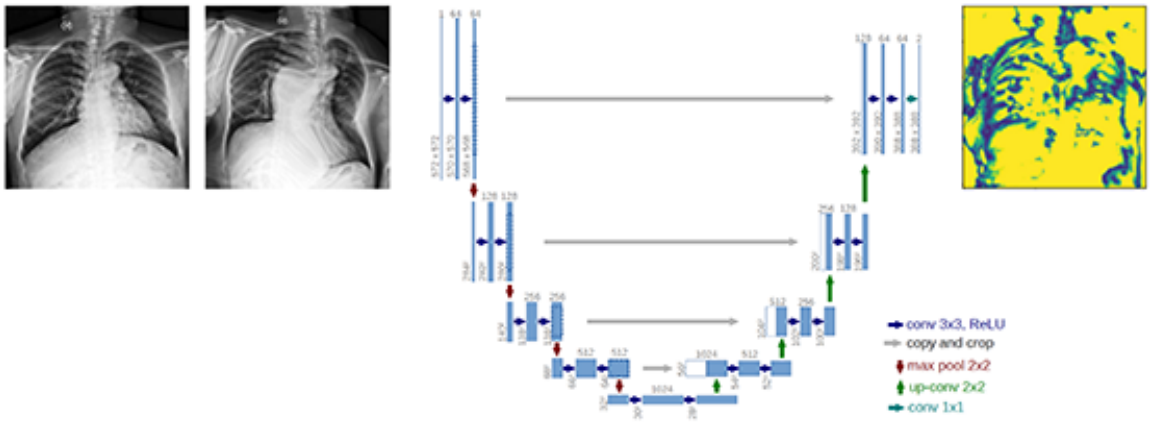
Create `val_dataset` and `val_loader`, corresponding to the `DataSet` and `DataLoader` for your validation set. The transforms can be the same as in the training set.

```
# Your code goes here
```

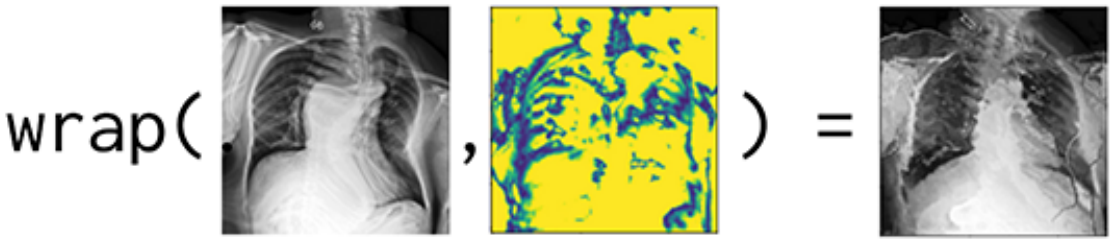
17.2.2 Model

As model, we'll use a U-Net. The input/output structure is quite different from what we've seen before:

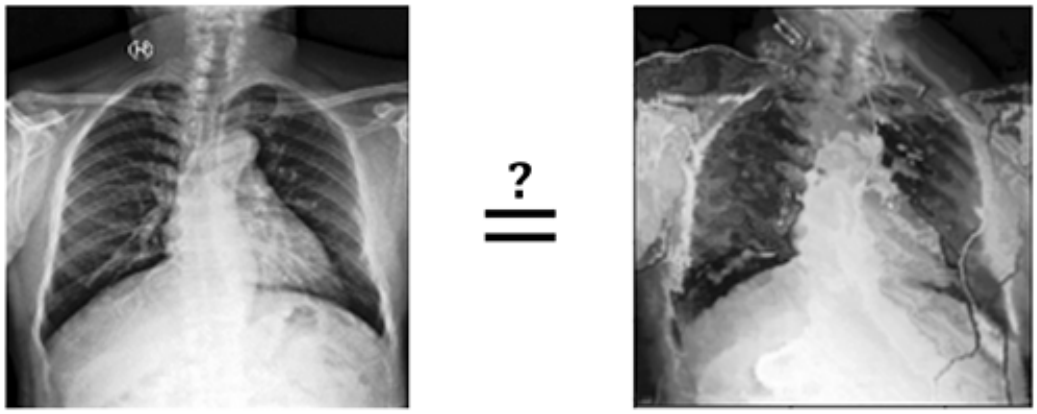
- the network takes as input two images: the *moving* and *fixed* images.
- it outputs one tensor representing the *deformation field*.



This *deformation field* can be applied to the *moving* image with the `monai.networks.blocks.Warp` block of Monai.



This deformed moving image is then compared to the *fixed* image: if they are similar, the deformation field is correctly registering the moving image on the fixed image. Keep in mind that this is done on **training** data, and we want the U-Net to learn to predict a proper deformation field given two new and unseen images. So we're not optimizing for a pair of images as would be done in conventional iterative registration, but training a model that can generalize.



Before starting, let's check that you can work on a GPU by runnning the following cell:

- if the device is "cuda" you are working on a GPU,
- if the device is "cpu" call a teacher.

```
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
    os.environ["PYTORCH_ENABLE_MPS_FALLBACK"]="1"
else:
    device = "cpu"
print(f'The used device is {device}')
```

Exercise

Construct a U-Net with suitable settings and name it `model`. Keep in mind that you want to be able to correctly apply its output to the input moving image with the `warp_layer`!

```
model = # FILL IN

warp_layer = monai.networks.blocks.Warp().to(device)
```

17.2.3 Objective function

We evaluate the similarity between the fixed image and the deformed moving image with the `MSELoss()`. The L1 or SSIM losses seen in the previous section could also be used. Furthermore, the deformation field is regularized with `BendingEnergyLoss`. This is a penalty that takes the smoothness of the deformation field into account: if it's not smooth enough, the bending energy is high. Thus, our model will favor smooth deformation fields.

Finally, we pick an optimizer, in this case again an Adam optimizer.

```
image_loss = torch.nn.MSELoss()
regularization = monai.losses.BendingEnergyLoss()
optimizer = torch.optim.Adam(model.parameters(), 1e-3)
```

Exercise

Add a learning rate scheduler that lowers the learning rate by a factor ten every 100 epochs.

```
# Your code goes here
```

To warp the moving image using the predicted deformation field and *then* compute the loss between the deformed image and the fixed image, we define a forward function which does all this. The output of this function is `pred_image`.

```
def forward(batch_data, model):
    """
    Applies the model to a batch of data.

    Args:
        batch_data (dict): a batch of samples computed by a DataLoader.
        model (Module): a model computing the deformation field.

    Returns:
        ddf (Tensor): batch of deformation fields.
        pred_image (Tensor): batch of deformed moving images.

    """
    fixed_image = batch_data["fixed"].to(device).float()
    moving_image = batch_data["moving"].to(device).float()

    # predict DDF
    ddf = model(torch.cat((moving_image, fixed_image), dim=1))

    # warp moving image and label with the predicted ddf
    pred_image = warp_layer(moving_image, ddf)

    return ddf, pred_image
```

You can supervise the training process in W&B, in which at each epoch a batch of validation images are used to compute the comparison images of your choice, based on the parameter `method`.

```
def log_to_wandb(epoch, train_loss, val_loss, pred_batch, fixed_batch, method=
↪"checkerboard"):
    """ Function that logs ongoing training variables to W&B """
    import skimage.util as skut

    log_imgs = []
    for fixed_pt, pred_pt in zip(pred_batch, fixed_batch):
        fixed_np = np.squeeze(fixed_pt.cpu().detach())
        pred_np = np.squeeze(pred_pt.cpu().detach())
        comp_checker = skut.compare_images(fixed_np, pred_np, method=method)
        log_imgs.append(wandb.Image(comp_checker))

    # Send epoch, losses and images to W&B
    wandb.log({'epoch': epoch, 'train_loss': train_loss, 'val_loss': val_loss,
↪'results': log_imgs})
```

(continues on next page)

(continued from previous page)

17.2.4 Training time

Use the following cells to train your network. You may choose different parameters to improve the performance!

```
# Choose your parameters

max_epochs = 200
reg_weight = 0 # By default 0, but you can investigate what it does
```

```
from tqdm import tqdm

run = wandb.init(
    project='tutorial4_registration',
    config={
        'lr': optimizer.param_groups[0]["lr"],
        'batch_size': train_loader.batch_size,
        'regularization': reg_weight,
        'loss_function': str(image_loss)
    }
)
# Do not hesitate to enrich this list of settings to be able to correctly keep track␣
↪of your experiments!
# For example you should add information on your model...

run_id = run.id # We remember here the run ID to be able to write the evaluation␣
↪metrics

for epoch in tqdm(range(max_epochs)):
    model.train()
    epoch_loss = 0
    for batch_data in train_loader:
        optimizer.zero_grad()

        ddf, pred_image = forward(batch_data, model)

        fixed_image = batch_data["fixed"].to(device).float()
        reg = regularization(ddf)
        loss = image_loss(pred_image, fixed_image) + reg_weight * reg
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    epoch_loss /= len(train_loader)

    model.eval()
    val_epoch_loss = 0
    for batch_data in val_loader:
        ddf, pred_image = forward(batch_data, model)
        fixed_image = batch_data["fixed"].to(device).float()
        reg = regularization(ddf)
        loss = image_loss(pred_image, fixed_image) + reg_weight * reg
        val_epoch_loss += loss.item()
```

(continues on next page)

(continued from previous page)

```
    val_epoch_loss /= len(val_loader)

    log_to_wandb(epoch, epoch_loss, val_epoch_loss, pred_image, fixed_image)

run.finish()
```

17.2.5 Evaluation of the trained model

Now that the model has been trained, it's time to evaluate its performance. Use the code below to visualize samples and deformation fields.

Exercise

Are you satisfied with these registration results? Do they seem anatomically plausible? Try out different regularization factors (`reg_weight`) and see what they do to the registration.

Answer:

```
def visualize_prediction(sample, model, method="checkerboard"):
    """
    Plot three images: fixed, moving and comparison.

    Args:
        sample (dict): sample of dataset created with `build_dataset`.
        model (Module): a model computing the deformation field.
        method (str): method used by `skimage.util.compare_image`.
    """
    import skimage.util as skut

    skut_methods = ["diff", "blend", "checkerboard"]
    if method not in skut_methods:
        raise ValueError(f"Method must be chosen in {skut_methods}.\n"
                         f"Current value is {method}.")

    model.eval()

    # Compute deformation field + deformed image
    batch_data = {
        "fixed": sample["fixed"].unsqueeze(0),
        "moving": sample["moving"].unsqueeze(0),
    }
    ddf, pred_image = forward(batch_data, model)
    ddf = ddf.detach().cpu().numpy().squeeze()
    ddf = np.linalg.norm(ddf, axis=0).squeeze()

    # Squeeze images
    fixed = np.squeeze(sample["fixed"])
    moving = np.squeeze(sample["moving"])
    deformed = np.squeeze(pred_image.detach().cpu())

    # Generate comparison image
    comp_checker = skut.compare_images(fixed, deformed, method=method, n_tiles=(4, 4))
```

(continues on next page)

(continued from previous page)

```
    # Plot everything
    fig, axs = plt.subplots(1, 5, figsize=(18, 5))
    axs[0].imshow(fixed, cmap='gray')
    axs[0].set_title('Fixed')
    axs[1].imshow(moving, cmap='gray')
    axs[1].set_title('Moving')
    axs[2].imshow(deformed, cmap='gray')
    axs[2].set_title('Deformed')
    axs[3].imshow(comp_checker, cmap='gray')
    axs[3].set_title('Comparison')
    dpl = axs[4].imshow(ddf, clim=(0, 10))
    fig.colorbar(dpl, ax=axs[4])
    plt.show()
    plt.show()
for sample in val_dataset:
    visualize_prediction(sample, model)
```

Exercise

Compute the Jacobian determinant at each image voxel. How many of these are negative? Can you improve upon this?

17.3 Part 2 - Equivariance

In this part, we are going to use some concepts that you've learned in the lecture on geometric deep learning. We are going to look at the equivariance properties of a neural network architecture that you should by now be very familiar with: the U-Net. We will again use the chest X-ray segmentation problem. Because training a network is not the focus here, we have pretrained a network that you can use for these experiments.

17.3.1 Data loading

We will again use the same utility functions as in Tutorial 3 to build a dictionary of files and load rib data.

```
import os
import numpy as np
import matplotlib.pyplot as plt
import glob
import monai
from PIL import Image
import torch

def build_dict_ribs(data_path, mode='train'):
    """
    This function returns a list of dictionaries, each dictionary containing the keys
↪'img' and 'mask'
    that returns the path to the corresponding image.

    Args:
        data_path (str): path to the root folder of the data set.
        mode (str): subset used. Must correspond to 'train', 'val' or 'test'.
```

(continues on next page)

(continued from previous page)

```
    Returns:
        (List[Dict[str, str]]) list of the dictionaries containing the paths of X-ray␣
↪images and masks.
    """
    # test if mode is correct
    if mode not in ["train", "val", "test"]:
        raise ValueError(f"Please choose a mode in ['train', 'val', 'test']. Current␣
↪mode is {mode}.")

    # define empty dictionary
    dicts = []
    # list all .png files in directory, including the path
    paths_xray = glob.glob(os.path.join(data_path, mode, 'img', '*.png'))
    # make a corresponding list for all the mask files
    for xray_path in paths_xray:
        if mode == 'test':
            suffix = 'val'
        else:
            suffix = mode
        # find the binary mask that belongs to the original image, based on indexing␣
↪in the filename
        image_index = os.path.split(xray_path)[1].split('_')[-1].split('.')[0]
        # define path to mask file based on this index and add to list of mask paths
        mask_path = os.path.join(data_path, mode, 'mask', f'VinDr_RibCXR_{suffix}_
↪{image_index}.png')
        if os.path.exists(mask_path):
            dicts.append({'img': xray_path, 'mask': mask_path})
    return dicts

class LoadRibData(monai.transforms.Transform):
    """
    This custom Monai transform loads the data from the rib segmentation dataset.
    Defining a custom transform is simple; just overwrite the __init__ function and __
↪call__ function.
    """
    def __init__(self, keys=None):
        pass

    def __call__(self, sample):
        image = Image.open(sample['img']).convert('L') # import as grayscale image
        image = np.array(image, dtype=np.uint8)
        mask = Image.open(sample['mask']).convert('L') # import as grayscale image
        mask = np.array(mask, dtype=np.uint8)
        # mask has value 255 on rib pixels. Convert to binary array
        mask[np.where(mask==255)] = 1
        return {'img': image, 'mask': mask, 'img_meta_dict': {'affine': np.eye(2)},
                'mask_meta_dict': {'affine': np.eye(2)}}
```

Use the cell below to make a validation loader with a single image. This is sufficient for the small experiment that you will perform.

```
validation_dict_list = build_dict_ribs(data_path, mode='val')
validation_transform = monai.transforms.Compose(
    [
        LoadRibData(),
        monai.transforms.EnsureChannelFirstd(keys=['img', 'mask'], channel_dim='no_
↪channel'),
```

(continues on next page)

(continued from previous page)

```
        monai.transforms.HistogramNormalized(keys=['img']),
        monai.transforms.ScaleIntensityd(keys=['img'], minv=0, maxv=1),
        monai.transforms.Zoomd(keys=['img', 'mask'], zoom=0.25, mode=['bilinear',
 ↪'nearest'], keep_size=False),
        # monai.transforms.RandSpatialCropd(keys=['img', 'mask'], roi_size=[384, 384],
 ↪ random_size=False)
        monai.transforms.SpatialCropd(keys=['img', 'mask'], roi_center=[300, 300],␣
 ↪roi_size=[384 + 64, 384])
    ]
)
validation_data = monai.data.CacheDataset([validation_dict_list[3]],␣
 ↪transform=validation_transform)
validation_loader = monai.data.DataLoader(validation_data, batch_size=1,␣
 ↪shuffle=False)
```

17.3.2 Loading a pretrained model

We have already trained a model for you, the parameters of which were shared in JupyterLab as well. **Note**: if you downloaded the data set yourself, the model should be in the same folder as the images. If you already downloaded the data set but not the model, the model file is available here.

```
!wget -O trainedUNet.pt https://surfdrive.surf.nl/files/index.php/s/613zrvr0RDYZDqp/
 ↪download
```

```
pretrained_file = path.join(data_path, "trainedUNet.pt")
```

Next, we initialize a standard U-Net architecture and load the parameters of the pretrained network using the `load_state_dict` function.

```
import torch
import monai
import random

# Check whether we're using a GPU
if torch.cuda.is_available():
    n_gpus = torch.cuda.device_count()  # Total number of GPUs
    gpu_idx = random.randint(0, n_gpus - 1)  # Random GPU index
    device = torch.device(f'cuda:{gpu_idx}')
    print('Using GPU: {}'.format(device))
else:
    device = torch.device('cpu')
    print('GPU not found. Using CPU.')

model = monai.networks.nets.UNet(
    spatial_dims=2,
    in_channels=1,
    out_channels=1,
    channels = (8, 16, 32, 64, 128),
    strides=(2, 2, 2, 2),
    num_res_units=2,
    dropout=0.5
).to(device)
```

(continues on next page)

(continued from previous page)

```
model.load_state_dict(torch.load(pretrained_file))
model.eval()
```

Let's use the pretrained network to segment (part of) our image. Run the cell below.

```
for sample in validation_loader:

    img = sample['img'][:, :, :384, :384]
    mask = sample['mask'][:, :, :384, :384]
    output_noshift = torch.sigmoid(model(img.to(device))).detach().cpu().numpy().
↪squeeze()

    fig, ax = plt.subplots(1,2, figsize = [12, 10])
    # Plot X-ray image
    ax[0].imshow(img.squeeze(), 'gray')
    # Plot ground truth
    mask = np.squeeze(mask)
    overlay_mask = np.ma.masked_where(mask == 0, mask == 1)
    ax[0].imshow(overlay_mask, 'Greens', alpha = 0.7, clim=[0,1], interpolation=
↪'nearest')
    ax[0].set_title('Ground truth')
    # Plot output
    overlay_output = np.ma.masked_where(output_noshift < 0.1, output_noshift > 0.99)
    ax[1].imshow(img.squeeze(), 'gray')
    ax[1].imshow(overlay_output.squeeze(), 'Reds', alpha = 0.7, clim=[0,1])
    ax[1].set_title('Prediction')
    plt.show()
```

As you can see, segmentation isn't perfect, but that's also not the goal of this exercise. What we are going to look into is the translation equivariance (**Lecture 8**) of the U-Net. That is: if you translate the image by d pixels, does the output also simply change by d pixels. Note that this is a nice feature to have for a segmentation network: in principle we'd want our network to give us the same label for a pixel regardless of where the image was cut. The image below visualizes this principle. For segmentation of the pixels in the orange square, it shouldn't matter if we provide the red square or the green square as input to the U-Net.

Exercise

What do you think will happen to the U-Net's prediction if we give it a slightly shifted version of the image as input?

Now we make a small script that performs the above experiment. First, we obtain the segmentation in the red box and we call this `output_noshift`. Then we shift the green box by an offset and each time obtain a segmentation in this box using the same model. We start small with a shift/offset of just a **single pixel**.

Exercise

Run the cell below and observe the outputs. Can you spot differences between the two segmentation masks?

```
offset = 1

for sample in validation_loader:
```

(continues on next page)

(continued from previous page)

```
    # Original image
    img = sample['img'][:, :, :384, :384]
    mask = sample['mask'][:, :, :384, :384]
    output_noshift = torch.sigmoid(model(img.to(device))).detach().cpu().numpy().
↪squeeze()

    # Plot X-ray image
    fig, ax = plt.subplots(1,2, figsize = [12, 10])
    ax[0].imshow(img.squeeze(), 'gray')
    # Plot ground truth
    mask = np.squeeze(mask)
    overlay_mask = np.ma.masked_where(mask == 0, mask == 1)
    ax[0].imshow(overlay_mask, 'Greens', alpha = 0.7, clim=[0,1], interpolation=
↪'nearest')
    ax[0].set_title('Ground truth')
    # Plot output
    overlay_output = np.ma.masked_where(output_noshift < 0.1, output_noshift >0.99)
    ax[1].imshow(img.squeeze(), 'gray')
    ax[1].imshow(overlay_output.squeeze(), 'Reds', alpha = 0.7, clim=[0,1])
    ax[1].set_title('Prediction')
    plt.show()

    # Shifted image
    img = sample['img'][:, :, offset:offset+384, :384]
    mask = sample['mask'][:, :, offset:offset+384, :384]
    output = torch.sigmoid(model(img.to(device))).detach().cpu().numpy().squeeze()

    # Plot X-ray image
    fig, ax = plt.subplots(1,2, figsize = [12, 10])
    ax[0].imshow(img.squeeze(), 'gray')
    # Plot ground truth
    mask = np.squeeze(mask)
    overlay_mask = np.ma.masked_where(mask == 0, mask == 1)
    ax[0].imshow(overlay_mask, 'Greens', alpha = 0.7, clim=[0,1], interpolation=
↪'nearest')
    ax[0].set_title('Ground truth shifted')
    # Plot output
    overlay_output = np.ma.masked_where(output < 0.1, output >0.99)
    ax[1].imshow(img.squeeze(), 'gray')
    ax[1].imshow(overlay_output.squeeze(), 'Reds', alpha = 0.7, clim=[0,1])
    ax[1].set_title('Prediction shifted')
    plt.show()
```

To highlight the differences between both segmentation masks a bit more, we make a difference image. We correct for the shift applied so that we're not comparing apples and oranges. The next cell shows the difference image between the original image and what we get when we process an image that is shifted by one pixel.

Exercise

Given these results, is a U-Net translation equivariant, invariant, or neither?

```
plt.figure(figsize=(6, 6))
diffout = output_noshift[offset:, :384] - output[:-offset, :384]
plt.imshow(diffout, cmap='seismic', clim=[-1, 1])
```

(continues on next page)

(continued from previous page)

```
plt.title('Offset {}'.format(offset))
plt.colorbar()
plt.show()
```

We can repeat this for larger offsets. Let's take offsets up to 64 pixels, and each time compute the difference between the original and shifted image, in a subimage that should be unaffected by the shift. We store the L1 norm of the difference image in an array `norms` and plot these as a function of offset.

Exercise

The resulting plot shows that the U-Net is equivariant for none of the translations. This is due to a combination of border effects and downsampling layers. However, the plot also shows a particular pattern, in which the norm *dips* every 16 pixels of offset. Can you explain this based on the U-Net architecture?

```
norms = []
offsets = []
plot_differences = False  # Set to True to plot difference images for every offset

img = sample['img'][:, :, :384, :384]
mask = sample['mask'][:, :, :384, :384]
output_noshift = torch.sigmoid(model(img.to(device))).detach().cpu().numpy().squeeze()

for offset in range(1, 65):
    for sample in validation_loader:
        img = sample['img'][:, :, offset:offset+384, :384]
        mask = sample['mask'][:, :, offset:offset+384, :384]

        output = torch.sigmoid(model(img.to(device))).detach().cpu().numpy().squeeze()

        diffout = (output_noshift[offset:, :384] - output[:-offset, :384])[100:284,␣
↪100:284]
        offsets.append(offset)
        norms.append(np.sum(np.abs(diffout)))
        if plot_differences:
            plt.figure()
            plt.imshow(diffout, cmap='seismic', clim=[-1, 1])
            plt.title(f"Offset {offset}")
            plt.colorbar()
            plt.show()

plt.figure()
plt.plot(offsets, norms)
plt.xlabel('Offset')
plt.ylabel('Difference')
plt.show()
```